Autonomous Invariant-Preserving Deadlock-Free Semantic Resource Arbitration and Epistemic Priority Inheritance for Multi-Agent Transactional Databases

Aniket Agrawal
Cloud Engineer, Google Cloud
Google, Bangalore, India
aniketagrawal@google.com

***Abstract*—The deployment of autonomous multi-agent swarms operating over shared enterprise transactional databases introduces a fundamental concurrency crisis. Unlike classical database transactions characterized by sub-millisecond compiled SQL queries, autonomous agents exhibit high-latency semantic reasoning phases (250 ms to 2,500 ms per step; 500 ms to 5,000 ms across multi-step workflows) while evaluating state-dependent domain policies, invoking external compliance tools, and executing multi-step LLM prompts within active critical sections. Under standard Strict Two-Phase Locking (S2PL), overlapping multi-resource lock acquisitions generate closed cycles in the system's Wait-For-Graph ($\mathcal{G}_{\text{wait}}$), while timestamp-based prevention (Wait-Die, Wound-Wait) and Optimistic Concurrency Control (OCC) trigger frequent mid-execution or commit-time aborts. Crucially, in multi-agent swarms, transaction aborts induce severe *livelock token thrashing*, as aborted agents re-execute stochastic planning paths and discard hundreds to thousands of already-billed LLM inference tokens.**

To resolve this systems bottleneck within a centralized shared-memory transactional lock coordinator or connection-pool gateway (motivated by single-leader OLTP lock tables such as a Google Cloud Spanner Paxos group leader, AlloyDB/PostgreSQL, or CockroachDB, and evaluated here over an in-memory transactional row-state mirror backed by a persistent SQLite Write-Ahead Logging audit ledger), we present the Epistemic Priority Ceiling Protocol (EPCP), an autonomous, invariant-preserving, provably deadlock-free concurrency control architecture. EPCP establishes a monotonically latched dynamic priority metric $\widetilde{\Pi}_i(t)$ incorporating agent task tree depth, accumulated LLM token investment with aging, and inverse epistemic entropy, paired with a static upper-envelope preemption level $\Pi_i^{\max}$ over exact 64-bit integer lexicographical priority tuples. By combining 64-partition intent-footprint priority ceilings ($\mathcal{F}(T_i) \subseteq \{1, \ldots, 64\}$) with incremental in-footprint row locking and a guarded preemptive admission invariant ($\Pi_i^{\text{eff}}(t) > C^*_{-A_i}(t; \mathcal{F}(T_i))$), EPCP serializes initial admission across intersecting footprints while allowing agents to dynamically discover Step-2 row keys during Step-1 LLM synthesis without upfront row-key pre-declaration or unisolated speculative pre-passes, release Step-1 locks and clear footprint ceilings in < 0.2 ms upon domain pre-check rejection, and execute disjoint footprints in full parallel concurrency. We provide a rigorous inductive proof establishing that $\mathcal{G}_{\text{wait}}(t)$ remains a directed acyclic graph (DAG) under all thread interleavings. We implemented EPCP natively on macOS Apple Silicon over an in-memory row-state mirror backed by a persistent SQLite WAL ledger and benchmarked sustained multi-threaded transactions over an enterprise banking and supply chain schema (10,000 Accounts, 5,000 Inventory Items) under heavy Zipfian access skew ($\theta = 0.85$) across sub-millisecond cryptographic HMAC-SHA256 stress tests, 500 ms–1,500 ms multi-step LLM reasoning benchmarks, 3-table dynamic footprint-expansion tests, and live Google Cloud Vertex AI `gemini-2.5-flash` multi-agent runs against S2PL, 2PL-Wait Promote, Wound-Wait, Wait-Die, OCC, Coarse Shard-Scoped C2PL, Fine-Grained Partition C2PL ($B_{\text{part}} = 64$), 64-Partition Canonical Ordered Locking ($B_{\text{part}} = 64$), and Row-Level Canonical Ordered Locking. Empirical results demonstrate that EPCP achieves a 0.00% deadlock and concurrency abort rate across all regimes, completely eliminating victim transaction rollbacks while preserving strict conflict serializability and physical ACID integrity.

***Index Terms*—Transactional Databases, Multi-Agent Concurrency Control, Deadlock Elimination, Priority Ceiling Protocol, Epistemic Uncertainty, Transaction Processing.**

I. Introduction

Enterprise cloud platforms are undergoing a paradigm shift from human-in-the-loop task execution to autonomous multi-agent systems. Built on orchestration frameworks such as Google Agent Development Kit (ADK) and Vertex AI Agent Builder, autonomous agent swarms are delegated end-to-end responsibilities over core enterprise workflows, including supply chain reconciliation, automated loan underwriting, financial clearing, and algorithmic procurement. In these architectures, multiple autonomous agents execute concurrently through a centralized shared-memory transactional lock coordinator or connection-pool gateway. While this gateway architecture is motivated by single-node or single-leader OLTP lock managers such as a Google Cloud Spanner Paxos group leader [6], AlloyDB/PostgreSQL, or CockroachDB (leaving decentralized cross-leader two-phase commit ceiling synchronization across geo-distributed consensus groups to future work), our empirical testbed evaluates the coordinator over a synchronized in-memory transactional row-state mirror backed by a persistent SQLite Write-Ahead Logging (WAL) audit ledger so lock-management primitives are isolated without external network variance.

A. The Semantic Reasoning Latency Expansion and Why Locks Span Inference

Classical transaction processing was developed under the assumption that transactions are short-lived, pre-compiled SQL queries with execution latencies on the order of 100 microseconds to 5 milliseconds [1], [2]. Under this microsecond regime, locking protocols such as Strict Two-Phase Locking (S2PL) [3] provide conflict serializability with negligible lock contention windows.

In autonomous multi-agent architectures, however, transactions are inherently interleaved with *state-dependent semantic reasoning loops*. A natural question is whether holding locks across LLM inference can be avoided altogether via (i) snapshot reads followed by short compare-and-swap (CAS) / Optimistic Concurrency Control (OCC) [12] validation at commit, (ii) simple commutative escrow columns (e.g., `reserved_amount`), or (iii) Saga-based non-isolated sub-transactions [10]. In regulated enterprise workflows, each of these alternatives breaks down under high contention:

1) **Failure of Snapshot-Read + CAS and Scalar Escrow on Non-Commutative State Mutations:** While simple scalar decrements can use commutative escrow counters, enterprise agent workflows perform *non-commutative, multi-attribute structural mutations*—such as corporate loan covenant restructuring, cross-collateralized margin re-indexing, and serialized lot compliance—where Step 2's multi-turn LLM synthesis, AML polynomial risk scoring, and generated SQL/JSON compliance patch depend nonlinearly on the exact vector of row attributes read in Step 1. Specifically, in our banking and inventory schemas, concurrent agents simultaneously read and mutate coupled high-contention attributes—including dynamic credit-tier covenants (`CustomerTier`, `CreditLimit`), net liquidity ratios (`Balance`), and monotonic audit counters (`AccountVersion`) in `Accounts`, alongside active reservation ratios (`StockCount`, `ReservedCount`) and regulatory lot categories (`Category`, `ItemVersion`) in `Inventory`—all of which directly parameterize Infer(ϕ)'s risk-tier classification and counterparty selection, preventing separation of read-mostly semantic attributes from hot scalar escrow fields. When an agent acquires a pessimistic lock on R_a, it first runs an immediate sub-microsecond scalar sufficiency pre-filter (`Biz-Rule Reject`) to fast-exit obviously depleted accounts or out-of-stock SKUs with zero wasted LLM tokens; for funded rows passing this pre-filter, the subsequent Infer(ϕ) phase evaluates the full multi-attribute state of R_a. Under snapshot-read + CAS (OCC) [12], if any contending agent modifies R_a during the 500 ms–5,000 ms multi-step reasoning window, the generated compliance patch is invalidated and CAS fails at commit time *after 100% of the multi-step LLM token budget has already been expended*. Holding a pessimistic lock across Infer(ϕ) protects funded rows from mid-inference invalidation.
2) **Prohibition of Uncommitted State Exposure:** Saga-based agent workflows [10] release locks early by committing intermediate sub-transactions and relying on reverse compensating actions upon failure. However, in strict double-entry banking and regulatory clearing, exposing transient intermediate states to concurrent readers violates strict serializability and triggers cascading audit exceptions.

Consequently, preserving strict ACID isolation without post-hoc validation rollbacks on non-commutative multi-attribute workflows requires holding pessimistic locks across intermediate reasoning phases, expanding lock retention durations from milliseconds to several seconds (500 ms to 5,000 ms)—a $10^3\times$ to $10^4\times$ duration expansion.

B. The Livelock and Token Thrashing Crisis

When agents acquire locks across multiple resources in dynamic orderings (e.g., Agent A_1 acquiring Account $\rightarrow$ Inventory while Agent A_2 acquires Inventory $\rightarrow$ Account), Wait-For-Graph ($\mathcal{G}_{\text{wait}}$) cycles form rapidly. Under S2PL, deadlock detection modules identify cycles and forcibly abort a "victim" transaction [1]. Similarly, classical timestamp-ordered deadlock prevention schemes [5], [13] avoid cycles by aborting transactions whenever timestamp inversions occur: under non-preemptive *Wait-Die*, a younger requester immediately "dies" (aborts) upon encountering an older lock holder while an older requester waits; under preemptive *Wound-Wait* (used in Google Cloud Spanner [6]), an older requester immediately "wounds" (preemptively aborts) a younger lock holder while a younger requester waits.

In traditional relational databases, aborting an in-memory SQL transaction incurs minimal penalty [13]. In multi-agent swarms, however, aborting an in-flight agent transaction—whether via S2PL deadlock victim selection, Wait-Die self-abort, Wound-Wait preemption [5], or OCC validation failure [12]—results in **catastrophic token thrashing**:

- **Token & Financial Waste:** An agent rolled back mid-workflow loses its in-flight transactional state, discarding hundreds to thousands of already-billed LLM inference tokens (e.g., on Gemini 2.5 Pro / Flash endpoints).
- **Stochastic Lock Retrying:** Upon restart, the agent re-evaluates prompts against mutated state, frequently re-colliding with contending agents.
- **System Saturation:** Under heavy partition skew, abort-based concurrency controllers suffer persistent rollback cascades.

Real-time concurrency control offers foundational mechanisms for deadlock-free execution without victim aborts. Priority Inheritance and Priority Ceiling Protocols (PCP) were pioneered by Sha, Rajkumar, and Lehoczky [4] for uniprocessor tasks with static priorities, and extended to dynamic-priority real-time tasks (such as Earliest Deadline First) by Chen and Lin's Dynamic Priority Ceiling Protocol [7] and Baker's Stack Resource Policy (SRP) [8], which decouple

static resource preemption levels from dynamic task urgency. In real-time databases, Abbott and Garcia-Molina [9] evaluated priority-cognizant 2PL variants—*2PL-High Priority* (which aborts lower-priority lock holders, inducing token waste) and *2PL-Wait Promote* (which applies priority inheritance upon direct row lock conflicts). Crucially, because *2PL-Wait Promote* lacks a preemptive partition priority ceiling admission gate, transactions still acquire initial locks in opposing orders and suffer multi-resource $\mathcal{G}_{\text{wait}}$ deadlocks. In parallel, classical database deadlock prevention via row-level Conservative 2PL (C2PL) [2], canonical ordered row locking, or deterministic scheduling such as Calvin [11] requires transactions to either pre-declare and lock every exact row key upfront or sort all target row IDs prior to acquiring any lock. In multi-step agentic workflows, however, the fine-grained target row key R_b in Step 2 (e.g., which specific warehouse lot or counterparty ledger row to debit) is discovered dynamically from Step 1's LLM inference $\text{Infer}(\phi)$, making exact row-level C2PL or upfront canonical row sorting impossible without an unisolated speculative discovery pre-pass.

It is important to separate two orthogonal design axes when routing headers expose a 64-partition intent footprint $\mathcal{F}(T_i) \subseteq \{1, \ldots, 64\}$: (i) the *partition-granularity concurrency gain* of decomposing coarse domain shards ($B_{\text{shard}} = 8$) into $B_{\text{part}} = 64$ intent hash buckets (metadata that Fine-Grained Partition C2PL or 64-Partition Canonical Ordered Locking can also exploit), and (ii) the *incremental row-level locking, epistemic priority inheritance, and dynamic expansion gains* unique to EPCP. Specifically, once R_a is acquired ($\text{Held}(A_i, t) \neq \emptyset$), EPCP's intersecting-footprint ceiling ($C^*_{-A_k}(t; \mathcal{F}(T_k))$) gates all partitions in $\mathcal{F}(T_i)$ (including Step 2's partition) against concurrent entry during Step-1 inference—matching the active-execution partition exclusion footprint of 64-partition C2PL (and on a < 0.2 ms `Biz-Rule Reject`, both release their held locks in < 0.2 ms). Relative to 64-partition C2PL and 64-partition Canonical Ordered Locking (both benchmarked directly in Section V), EPCP's architectural distinction lies along three dimensions: (a) *incremental fine-grained row locking within* $\mathcal{F}(T_i)$ without requiring upfront row keys or unisolated speculative discovery pre-passes, (b) *transitive epistemic priority inheritance and live* $\tilde{\Pi}_i^{\text{eff}}(t)$ *wake-up ordering* (which dispatches waiting agents with the highest accumulated token investment and action certainty first rather than FIFO or static-priority partition queues, and further allows finite-stock workloads to prioritize deeper high-quantity settlements), and (c) *seamless mid-flight out-of-footprint partition expansion* ($\text{part}(R_{\text{req}}) \notin \mathcal{F}(T_i)$) with SHA-256 `state_version_digest` context-cache replay. Finally, while compensating workflow managers such as SagaLLM [10] relax serializability via non-isolated sub-transactions and coarse workload-level adaptive schemes that switch whole transactions between pure OCC [12] and pessimistic locking [13] remain bounded by the pure OCC and 2PL regimes evaluated in Section V under stationary high skew ($\theta = 0.85$), neither provides a provably deadlock-free, zero-abort priority ceiling protocol coupled to live LLM token investment and epistemic action entropy.

C. Contributions

In this work, we present the **Epistemic Priority Ceiling Protocol (EPCP)**, an autonomous, invariant-preserving (in the concurrency-control sense of maintaining cycle-free wait invariants dynamically without background deadlock detectors or victim aborts) concurrency control architecture engineered for high-latency semantic agent transactions. Our primary contributions are:

1) **Dual-Level Epistemic Priority Architecture ($\Pi_i^{\max}$ and $\tilde{\Pi}_i(t)$):** A formal priority formulation coupling task delegation depth (d_i), clipped cumulative token and aging expenditure ($\Omega_i(t)$), and inverse normalized token-level Shannon action entropy ($1 - \mathcal{H}_i(t)$, where $\mathcal{H}_i(t) \in [0, 1]$ measures action distribution uncertainty), represented as exact 64-bit integer lexicographical tuples to guarantee strict total ordering without floating-point precision loss.
2) **Intersecting-Footprint Partition Priority Ceilings ($C^*_{-A_i}(t; \mathcal{F}(T_i))$):** A 64-partition lock admission invariant where an agent declares an ADK routing-header hash footprint $\mathcal{F}(T_i) \subseteq \{1, \ldots, 64\}$ at registration, serializing initial entry across intersecting dynamic footprints ($\mathcal{F}(T_i) \cap \mathcal{F}(T_k) \neq \emptyset$) to prevent Step-2 dynamic-discovery cycles while executing disjoint footprints ($\mathcal{F}(T_i) \cap \mathcal{F}(T_k) = \emptyset$) in parallel concurrency. We explicitly analyze the false-sharing trade-off of $B_{\text{part}} = 64$ (balancing single-register `uint64_t` $O(1)$ bitwise-`AND` efficiency against hash-collision blocking across disjoint rows during inference windows) and isolate EPCP's epistemic scheduling gains from static 64-bucket partitioning.
3) **Provable Deadlock Immunity (Theorem 1):** A rigorous inductive proof over unified contention and ceiling wait edges demonstrating that $\mathcal{G}_{\text{wait}}(t)$ remains a directed acyclic graph (DAG) at all times, guaranteeing a 0.00% deadlock abort rate without victim selection.
4) **Bounded Priority Inversion & Starvation Freedom (Theorems 2 & 3):** Proofs establishing single-critical-section inversion bounds per disjoint partition component via transitive priority inheritance and staggered-arrival gating, plus unconditional finite-time completion via timed and release-refreshed senior saturation promotion.
5) **Comprehensive Empirical & Live Vertex AI Validation:** Multi-threaded benchmarks implemented over an in-memory transactional row-state mirror backed by a persistent SQLite WAL audit ledger on an enterprise banking and supply chain schema (10,000 Accounts, 5,000 Inventory Items) under Zipfian skew ($\theta = 0.85$), spanning sub-millisecond HMAC-SHA256 stress tests (with Zero-Depletion Controls across $K \in \{20, 30, 40\}$), 500 ms–1,500 ms 2-step LLM reasoning benchmarks across 11 protocols and ablations [2], [3], [5], [9], [12], [13], 3-table ($|\mathcal{F}(T_i)| = 3$) dynamic out-of-footprint

expansion evaluations, and live Google Cloud Vertex AI `gemini-2.5-flash` multi-agent evaluations on a dedicated enterprise Google Cloud Vertex AI environment.

II. System Model and Problem Formulation

A. Execution Environment, False-Sharing Trade-offs, and Intent-Scoped Footprints

We consider a transactional database system $\mathcal{D}$ arbitrated by a centralized shared-memory lock coordinator (motivated by a single Spanner Paxos group leader's local lock table [6] and implemented over an in-memory committed row-state mirror backed by SQLite WAL) maintaining a set of shared row-level resources $\mathcal{R} = \{R_1, R_2, \ldots, R_M\}$, mapped via a deterministic FNV-1a 64-bit hash function $\text{part} : \mathcal{R} \to \{1, \ldots, B_{\text{part}}\}$ ($B_{\text{part}} = 64$) into fine-grained logical lock partitions. Each transaction's partition footprint is represented compactly as a single 64-bit unsigned integer bitmask (`uint64_t`), enabling $O(1)$ hardware bitwise-`AND` footprint intersection checks. A swarm of K autonomous agent worker threads $\mathcal{A} = \{A_1, A_2, \ldots, A_K\}$ concurrently execute transactional tasks.

Unlike row-level C2PL (which requires knowing every exact row primary key R_a, R_b before locking), EPCP requires only an **Intent-Scoped Partition Footprint** $\mathcal{F}(T_i) \subseteq \{1, \ldots, B_{\text{part}}\}$ (typically $|\mathcal{F}(T_i)| \in \{2, 3\}$ out of $B_{\text{part}} = 64$), extracted statically from the mandatory routing metadata of the agent's ADK/MCP tool invocation schema (e.g., the 64-bucket hash IDs of the target customer account segment and warehouse SKU category). Mapping 10,000 accounts and 5,000 inventory items onto $B_{\text{part}} = 64$ partitions introduces a deliberate architectural trade-off between hardware bitmask efficiency and *false-sharing partition contention*: two transactions targeting completely disjoint physical rows ($R_a \neq R_{a'}$ and $R_b \neq R_{b'}$) that hash to a common bucket in $\{1, \ldots, 64\}$ ($1/64 \approx 1.56\%$ single-bucket collision probability) have intersecting footprints ($\mathcal{F}(T_i) \cap \mathcal{F}(T_k) \neq \emptyset$), causing the second transaction to wait at EPCP's admission gate during the first transaction's reasoning window before acquiring $R_{a'}$ or running its `Biz-Rule Reject` pre-filter, whereas row-level S2PL admits disjoint rows within the same hash bucket concurrently. We select $B_{\text{part}} = 64$ because a 64-bit word fits in a single native CPU register while keeping random two-partition overlap low enough ($\approx 6.1\%$ for uniform pairs) that eliminating multi-resource deadlocks and token rollbacks yields $+20.3\%$– $+ 22.9\%$ higher committed goodput than row-level S2PL under heavy Zipfian contention ($\theta = 0.85$); deployments requiring still lower false sharing can widen footprints to 256-bit AVX2 or 512-bit AVX-512 bitvectors ($B_{\text{part}} \in \{256, 512\}$) in $O(1)$ SIMD instructions.

While $\mathcal{F}(T_i)$ is known at registration, the exact fine-grained row keys $R_a, R_b \in \mathcal{R}$ within $\mathcal{F}(T_i)$ are discovered and locked incrementally during execution:

$$T_i = \langle \text{Register}(\mathcal{F}(T_i)),\ \text{Lock}(R_a),\ \text{BizCheck}(R_a),\ \text{Infer}(\phi),\ \text{StageWrite}(R_a),\ \text{Lock}(R_b),\ \ldots,\ \text{Commit} \rangle \tag{1}$$

where $\text{Infer}(\phi)$ denotes parameterized LLM policy inference (using ϕ to distinguish model parameters from the Zipfian skew parameter θ), and $\text{StageWrite}(R_a)$ stages mutations in a transaction-local write buffer that is only flushed atomically to physical database storage at Commit after all locks are held. If an agent ever attempts to request an out-of-footprint resource R_{req} with $\text{part}(R_{\text{req}}) \notin \mathcal{F}(T_i)$, the coordinator rejects the lock call (returning RESTART in Algorithm 1), discards the uncommitted local write buffer (so zero physical database rollback is required), memoizes completed $\text{Infer}(\phi)$ outputs in the agent's local context cache keyed by a cryptographic SHA-256 digest of the read attributes of R_a (`state_version_digest`), releases all held locks with full waiter wake-up while keeping $A_i \in \mathcal{A}_{\text{active}}(t)$, expands $\mathcal{F}(T_i) \leftarrow \mathcal{F}(T_i) \cup \{\text{part}(R_{\text{req}})\}$ while preserving $(t_{\text{start},i}, \text{rank}(T_i))$, and unwinds execution to restart from Step 1 ($\text{Lock}(R_a)$). Upon re-acquiring R_a, if R_a's read attributes are unchanged (matching `state_version_digest`), the memoized inference output is replayed in microseconds with **zero re-billed LLM tokens**; if R_a was modified by another transaction during the brief unlock window, the SHA-256 digest misses and $\text{Infer}(\phi)$ is freshly re-evaluated against the new state. Thus, EPCP's zero-token-waste guarantee holds unconditionally whenever declared footprints bound candidate partitions upfront (as in our 2-table ADK schema), and holds conditionally on `state_version_digest` cache hits (91.4% hit rate at 10% expansion in our 3-table expansion benchmark in Section V-B) when dynamic footprint expansion occurs.

B. Unified Wait-For-Graph and Deadlock Formalization

Let $\text{Held}(A_j, t) \subseteq \mathcal{R}$ denote the set of resources currently locked by agent A_j at time t.

Definition 1 (Unified Wait-For-Graph)**.** *The Unified Wait-For-Graph at time t, denoted $\mathcal{G}_{wait}(t) = (\mathcal{V}, \mathcal{E}(t))$, is a directed graph where vertices $\mathcal{V} = \mathcal{A}$ represent active agent worker threads, and a directed edge $(A_i \to A_j) \in \mathcal{E}(t)$ $(i \neq j)$ exists if and only if agent A_i is currently suspended awaiting lock acquisition on resource R_{req} ($\text{part}(R_{req}) \in \mathcal{F}(T_i)$) because:*

1) ***Direct Contention Wait:*** *$R_{req} \in \text{Held}(A_j, t)$; or*
2) ***Ceiling Preemption Wait:*** *Eq. (12) fails for A_i, and $A_j \in \mathcal{B}_{ceil}(A_i, t)$ $(j \neq i)$ is an active lock holder ($\text{Held}(A_j, t) \neq \emptyset$) blocking A_i's admission under Section III-C.*

Definition 2 (Deadlock State)**.** *A system is in a deadlock state at time t if and only if there exists a directed cycle in $\mathcal{G}_{wait}(t)$:*

$$A_{p_1} \to A_{p_2} \to \cdots \to A_{p_L} \to A_{p_1} \tag{2}$$

Under S2PL, resolving this cycle requires selecting a victim agent A_{p_v} and aborting its transaction, rolling back uncommitted writes and discarding in-flight LLM reasoning tokens.

III. The Epistemic Priority Ceiling Protocol

A. Monotonically Latched Epistemic Priority Formulation

Unlike classical real-time operating systems where thread priorities are static integers [4], [8], autonomous agent priorities must reflect both the economic sunk cost and epistemic certainty of in-flight reasoning tasks while remaining strictly ordered and non-decreasing during an active transaction. Following Baker's SRP principle [8], EPCP maintains two complementary priority quantities per transaction T_i: (1) a static **Transaction Upper-Envelope Preemption Level** $\Pi_i^{\max}$ (determined at registration by task depth d_i and arrival rank $\text{rank}(T_i)$, and upgraded only upon senior saturation horizon T_{sat}) that defines partition ceilings so ceilings never shift mid-lock and cause self-deadlock, and (2) a live **Monotonically Latched Dynamic Priority** $\tilde{\Pi}_i(t)$ (driven by real-time token expenditure $\text{Tokens}_i(t)$, wait-time aging, and action certainty $1-\mathcal{H}_i(t)$) that orders condition-variable wake-up queues and dynamic urgency scheduling among waiting agents. Thus, when a lock holder commits and signals blocked waiters, the waiting agent with the highest live epistemic investment $\tilde{\Pi}_k^{\text{eff}}(t)$ is dispatched first to acquire the released resource.

Definition 3 (Dynamic and Latched Epistemic Priority)**.** *At time t, the instantaneous epistemic score $\Pi_i(t) \in \mathbb{R}^+$ of agent A_i executing transaction T_i (started at $t_{start,i}$) is:*

$$\Pi_i(t) \triangleq \Pi_{base} + w_d \cdot d_i + w_\Omega \cdot \ln(1+\Omega_i(t)) + w_c \cdot (1-\mathcal{H}_i(t)) \quad (3)$$

where $\Pi_{base} = 1.0$, $d_i \in \{1, \ldots, d_{\max}\}$ ($d_{\max} = 4$) is the agent call-graph delegation depth, and $\Omega_i(t)$ combines cumulative LLM token investment with linear wait-time aging ($\gamma_{age} = 0.05\ s^{-1}$, explicitly clipped at $\Omega_{\max} = 10^4$):

$$\Omega_i(t) \triangleq \min\left(\Omega_{\max},\ \frac{Tokens_i(t)}{100} + \gamma_{age}(t - t_{start,i})\right) \quad (4)$$

with $\Omega_i(t) \in [0, \Omega_{\max}]$. The normalized token-level Shannon entropy $\mathcal{H}_i(t) \in [0,1]$ over the agent's candidate tool-action distribution $p_{i,t}(v)$ ($v \in \mathcal{V}_{act}$, extracted from the softmax log-probabilities of the LLM's structured tool-call head at time t, initialized to neutral uncertainty $\mathcal{H}_i(t_{start,i}) = 0.5$ prior to the first inference step) is defined as:

$$\mathcal{H}_i(t) \triangleq \begin{cases} 0, & \text{if } |\mathcal{V}_{act}| \le 1, \\ -\frac{1}{\ln|\mathcal{V}_{act}|} \sum_{v \in \mathcal{V}_{act}} p_{i,t}(v) \ln p_{i,t}(v), & \text{if } |\mathcal{V}_{act}| > 1, \end{cases} \quad (5)$$

with weights fixed at $w_d = 2.0, w_\Omega = 1.5, w_c = 3.0$ (chosen so one delegation level $w_d = 2.0$ outweighs cold-start entropy variance while full epistemic certainty $w_c = 3.0$ and log-token investment $w_\Omega \ln(1+\Omega_i(t)) \in [0, 13.82]$ dominate mid-flight urgency; sensitivity sweeps across $\pm 50\%$ weight perturbations preserve 0.00% aborts with $< 2.1\%$ goodput variation). Furthermore, to guarantee unconditional starvation freedom across arbitrary depths (Theorem 3), if a waiting transaction's age reaches the senior saturation horizon T_{sat} (defaulting to $\Omega_{\max}/\gamma_{age} = 2 \times 10^5$ s as an asymptotic starvation safeguard, or configurable via `senior_saturation_sec` as an independent wall-clock SLA horizon such as $T_{sat} \in [5\ s, 60\ s]$ decoupled from $\Omega_{\max}$), its latched score AND its upper-envelope preemption score are atomically promoted to the uniform saturation ceiling $\Pi_{sat} \triangleq \Pi_{base} + w_d \cdot d_{\max} + w_\Omega \cdot \ln(1+\Omega_{\max}) + w_c$, updating $\Pi(P_m, t)$ for all $P_m \in \mathcal{F}(T_i)$ and immediately propagating the promoted priority to any blocking holders via Eqs. (13)–(14).

*Let $\Pi_i^{score}(t) \triangleq \max_{\tau \in [t_{start,i}, t]} \Pi_i(\tau)$ denote the monotonically latched dynamic score and $\Pi_{i,\max}^{score} \triangleq \Pi_{base} + w_d d_i + w_\Omega \ln(1+\Omega_{\max}) + w_c$ (upgraded to Π_{sat} if $t - t_{start,i} \ge T_{sat}$) denote the upper-envelope preemption score. To eliminate IEEE-754 floating-point mantissa precision loss or scalar underflow as the monotonic arrival sequence ID $\text{rank}(T_i) \in \mathbb{N}$ grows over unbounded non-idle streams, the coordinator represents and compares priorities internally as exact 64-bit integer lexicographical pairs $\tilde{\mathbf{\Pi}}_i(t) = \left(\lfloor 10^6 \Pi_i^{score}(t) \rfloor, -rank(T_i)\right)$ and $\mathbf{\Pi}_i^{\max} = \left(\lfloor 10^6 \Pi_{i,\max}^{score} \rfloor, -rank(T_i)\right) \in \mathbb{Z}_{\ge 0} \times \mathbb{Z}_{<0}$ (and rebases $rank(T_i) \leftarrow 1$ only when $\mathcal{A}_{active}(t) = \emptyset$; expanding transactions remain in $\mathcal{A}_{active}(t)$ during re-registration so $rank(T_i)$ is never rebased mid-flight). In scalar notation, we write this strict lexicographical tuple order symbolically via the tie-breaker $\epsilon_{tie} = 10^{-7}$ (where $\Pi^{score} - \epsilon_{tie} rank(T_i)$ is compact symbolic shorthand for the exact 64-bit integer tuple comparison $(\lfloor 10^6 \Pi^{score} \rfloor, -rank(T_i))$, rather than literal floating-point subtraction) as the **Monotonically Latched Dynamic Priority** $\tilde{\Pi}_i(t)$ and the **Transaction Upper-Envelope Preemption Level** $\Pi_i^{\max}$:*

$$\tilde{\Pi}_i(t) \triangleq \Pi_i^{score}(t) - \epsilon_{tie} \cdot rank(T_i), \quad (6)$$

$$\Pi_i^{\max} \triangleq \Pi_{i,\max}^{score} - \epsilon_{tie} \cdot rank(T_i) \quad (7)$$

By construction, for all $t_1 \le t_2$ during T_i's execution, $0 < \tilde{\Pi}_i(t_1) \le \tilde{\Pi}_i(t_2) \le \Pi_i^{\max}$, and $\Pi_i^{\max} \ne \Pi_j^{\max}$ for all $i \ne j$.

B. Atomic Registration and Intersecting-Footprint Partition Ceilings

Let $\mathcal{A}_{\text{active}}(t)$ denote the set of registered active agent threads at time t. In EPCP, Register($\mathcal{F}(T_q)$) and A_q's initial lock check $\text{Lock}(R_a)$ execute as a **single atomic critical section under the coordinator mutex**: when A_q (executing T_q) arrives at time t_{arr} with declared footprint $\mathcal{F}(T_q)$, A_q is inserted into $\mathcal{A}_{\text{active}}(t)$ and into $\mathcal{U}(P_m, t) \triangleq \{A_r \in \mathcal{A}_{\text{active}}(t) \mid P_m \in \mathcal{F}(T_r)\}$ for all $P_m \in \mathcal{F}(T_q)$ (elevating each partition ceiling $\Pi(P_m, t)$ via the exact 64-bit integer tuple comparison $\mathbf{\Pi}_q^{\max}$), and immediately evaluates Eq. (12) for R_a. If Eq. (12) fails because an active lock holder A_k ($\text{Held}(A_k, t_{\text{arr}}) \ne \emptyset$) has an intersecting footprint $\mathcal{F}(T_k) \cap \mathcal{F}(T_q) \ne \emptyset$ (or blocks a higher-priority waiter on $\mathcal{F}(T_q)$), A_q atomically binds wait edges $(A_q \to A_k)$ for all $A_k \in \mathcal{B}_{\text{ceil}}(A_q, t_{\text{arr}})$ (defined in Section III-C) and propagates $\Pi_q^{\max}$ to A_k via transitive priority inheritance (Eqs. (13)–(14)) before releasing the coordinator mutex to sleep on its condition variable $\text{CV}(A_q)$.

Definition 4 (Partition Ceiling and Intersecting-Footprint System Ceiling)**.** *For each logical partition $P_m \in \{1, \ldots, B_{part}\}$, the partition priority ceiling $\Pi(P_m, t)$ (and resource ceiling*

$C(R_j,t) \triangleq \Pi(\textit{part}(R_j),t))$ *is the maximum upper-envelope preemption bound among all active agents registered for partition* P_m*:*

$$\Pi(P_m,t) \triangleq \max\Big(\{0\} \cup \big\{\Pi_q^{\max} \;\big|\; A_q \in \mathcal{U}(P_m,t)\big\}\Big) \quad (8)$$

Because $\Pi(P_m,t)$ *is computed over the upper-envelope* $\Pi_q^{\max}$ *of registered agents,* $\Pi(P_m,t) \geq \tilde{\Pi}_q(\tau)$ *holds for every* $A_q \in \mathcal{U}(P_m,t)$ *at all instants* τ*. For an agent* A_i *with footprint* $\mathcal{F}(T_i)$*, let* $\mathcal{K}_{\textit{ext}}^{\textit{hold}}(A_i,t) \triangleq \{A_k \in \mathcal{A}_{\textit{active}}(t) \setminus \{A_i\} \mid \textit{Held}(A_k,t) \neq \emptyset \;\wedge\; \mathcal{F}(T_k) \cap \mathcal{F}(T_i) \neq \emptyset\}$ *denote the set of active external lock-holding agents whose declared partition footprint intersects* $\mathcal{F}(T_i)$*. Furthermore, to prevent a newly arriving lower-priority transaction from cutting ahead of a higher-priority (or* $T_{\textit{sat}}$*-promoted) multi-partition waiter* A_q *queued at the admission gate—including during the Mesa condition-variable wake-up handoff window between* RELEASEALLLOCKS *clearing incoming wait edges and the woken waiter* A_q *re-acquiring the coordinator mutex—let* $\mathcal{K}_{\textit{ext}}^{\textit{queue}}(A_i,t) \triangleq \{A_k \in \mathcal{A}_{\textit{active}}(t) \setminus \{A_i\} \mid \textit{Held}(A_i,t) = \emptyset \;\wedge\; \textit{Held}(A_k,t) \neq \emptyset \;\wedge\; \exists A_q \in \textit{Waiters}(A_k,t) \text{ s.t. } \mathcal{F}(T_q) \cap \mathcal{F}(T_i) \neq \emptyset \wedge (\boldsymbol{\Pi}_q^{\textit{eff}}(t) > \boldsymbol{\Pi}_i^{\textit{eff}}(t) \vee \tilde{\boldsymbol{\Pi}}_q^{\textit{eff}}(t) > \tilde{\boldsymbol{\Pi}}_i^{\textit{eff}}(t))\}$ *(where* $\Pi^{\textit{eff}}(t)$ *and* $\tilde{\Pi}^{\textit{eff}}(t)$ *are defined in Eqs.* (13)–(14)*) denote the active lock holders blocking a higher-priority waiter on an intersecting partition, let* $\mathcal{W}_{\textit{pend}}(A_i,t) \triangleq \{A_q \in \mathcal{A}_{\textit{active}}(t) \setminus \{A_i\} \mid \textit{Held}(A_i,t) = \emptyset \;\wedge\; \textit{Held}(A_q,t) = \emptyset \;\wedge\; A_q \text{ is queued at } \textit{CV}(A_q) \;\wedge\; \mathcal{F}(T_q) \cap \mathcal{F}(T_i) \neq \emptyset \wedge (\boldsymbol{\Pi}_q^{\textit{eff}}(t) > \boldsymbol{\Pi}_i^{\textit{eff}}(t) \vee \tilde{\boldsymbol{\Pi}}_q^{\textit{eff}}(t) > \tilde{\boldsymbol{\Pi}}_i^{\textit{eff}}(t))\}$ *denote higher-priority pending waiters on an intersecting partition, and define* $\mathcal{K}_{\textit{ext}}(A_i,t) \triangleq \mathcal{K}_{\textit{ext}}^{\textit{hold}}(A_i,t) \cup \mathcal{K}_{\textit{ext}}^{\textit{queue}}(A_i,t)$*. The* ***External Intersecting-Footprint System Ceiling*** $C^*_{-A_i}(t;\mathcal{F}(T_i))$ *and the* ***Active-Holder Intersecting-Footprint System Ceiling*** $C^*(t;\mathcal{F}(T_i))$ *are defined as:*

$$\begin{aligned} C^*_{-A_i}(t;\mathcal{F}(T_i)) \triangleq \max\Big(&\{0\} \\ &\cup \big\{\Pi(P_m,t) \;\big|\; A_k \in \mathcal{K}_{\textit{ext}}^{\textit{hold}}(A_i,t), \\ &\quad P_m \in \mathcal{F}(T_i) \cap \mathcal{F}(T_k)\big\} \\ &\cup \big\{\Pi(P_m,t) \;\big|\; \textit{Held}(A_i,t) = \emptyset, \\ &\quad \exists A_k \in \mathcal{K}_{\textit{ext}}^{\textit{queue}}(A_i,t), \\ &\quad \exists A_q \in \textit{Waiters}(A_k,t) \;\vee\; A_q \in \mathcal{W}_{\textit{pend}}(A_i,t), \\ &\quad P_m \in \mathcal{F}(T_i) \cap \mathcal{F}(T_q)\big\}\Big), \end{aligned} \quad (9)$$

$$\begin{aligned} C^*(t;\mathcal{F}(T_i)) \triangleq \max\Big(&\{0\} \\ &\cup \big\{\Pi_k^{\textit{eff}}(t) \;\big|\; A_k \in \mathcal{A}_{\textit{active}}(t), \\ &\quad \textit{Held}(A_k,t) \neq \emptyset, \\ &\quad \mathcal{F}(T_i) \cap \mathcal{F}(T_k) \neq \emptyset\big\}\Big) \end{aligned} \quad (10)$$

Notice three fundamental properties of Eqs. (9)–(10)*:*

1) ***Parallel Concurrency Across Disjoint 64-Bucket Footprints*** *(*$\mathcal{K}_{\textit{ext}}(A_i,t) = \emptyset \wedge \mathcal{W}_{\textit{pend}}(A_i,t) = \emptyset$*): Whenever no external lock-holding transaction (or higher-priority queued waiter) has a footprint intersecting* $\mathcal{F}(T_i)$*, Eq.* (9) *takes the maximum over the empty set and yields* $C^*_{-A_i}(t;\mathcal{F}(T_i)) = 0 < \Pi_i^{\textit{eff}}(t)$*. Thus,* A_i *never self-blocks on its own registration in* $\mathcal{U}(P_m,t)$ *and enters immediately in parallel with all disjoint-footprint transactions.*
2) ***Intersecting-Footprint Admission Exclusion*** *(*$\mathcal{K}_{\textit{ext}}(A_i,t) \neq \emptyset$*): Because* $A_i \in \mathcal{U}(P_m,t)$ *for all* $P_m \in \mathcal{F}(T_i)$*, whenever an external transaction* $A_k \in \mathcal{K}_{\textit{ext}}^{\textit{hold}}(A_i,t)$ *already holds any lock (*$\textit{Held}(A_k,t) \neq \emptyset$*) and shares at least one partition* $P_{\textit{shared}} \in \mathcal{F}(T_i) \cap \mathcal{F}(T_k)$*, Eq.* (9) *yields* $C^*_{-A_i}(t;\mathcal{F}(T_i)) \geq \Pi(P_{\textit{shared}},t) \geq \max(\Pi_i^{\max},\Pi_k^{\max}) \geq \Pi_i^{\max}$*. Because Step-2 row keys inside* $\mathcal{F}(T_i)$ *are discovered dynamically during Step-1 LLM inference, this enforces* ***single-holder admission exclusion across intersecting footprints****: a newly arriving* A_i *(*$\textit{Held}(A_i,t) = \emptyset$*, even if* $\Pi_i^{\max} > \Pi_k^{\max}$*) is suspended at the admission gate on* $\textit{CV}(A_i)$ *rather than entering* $P_{\textit{shared}}$ *concurrently with* A_k *(which would risk* A_i *and* A_k *dynamically discovering and requesting each other's held rows in Step 2), and instead immediately elevates* A_k *via transitive priority inheritance (*$\Pi_k^{\textit{eff}}(t) \leftarrow \max(\Pi_k^{\textit{eff}}(t),\Pi_i^{\max})$*) so* A_k *completes its remaining steps rapidly and hands the footprint to* A_i*.*
3) ***Unconditional Step-2 Pass-Through for Active Lock Holders (Held****(*$A_i,t) \neq \emptyset$*):*** *Because footprints never expand while locks are held (Algorithm 1 Line 6 releases all held locks prior to footprint expansion), single-holder admission exclusion across intersecting footprints implies by induction over lock-acquisition events that active lock holders have pairwise disjoint footprints at all times* t*:*

$$\begin{aligned} \textit{Held}(A_i,t) \neq \emptyset \;&\Longrightarrow\; \mathcal{K}_{\textit{ext}}(A_i,t) = \emptyset \\ &\Longrightarrow\; C^*_{-A_i}(t;\mathcal{F}(T_i)) = 0 < \Pi_i^{\textit{eff}}(t) \\ &\quad\wedge\; C^*(t;\mathcal{F}(T_i)) = \Pi_i^{\textit{eff}}(t). \end{aligned} \quad (11)$$

Thus, once A_i *holds its Step-1 lock (*$\textit{Held}(A_i,t) \neq \emptyset$*), both disjuncts of Eq.* (12) *are satisfied simultaneously when* A_i *requests its Step-2 locks within* $\mathcal{F}(T_i)$*, guaranteeing zero secondary ceiling self-blocking even if an unadmitted multi-partition waiter* A_M *is registered on one of* $\mathcal{F}(T_i)$*'s partitions while blocked on a disjoint lock holder.*

C. EPCP Resource Allocation and Guarded Pass-Through Rules

When agent A_i requests lock acquisition on resource R_{req} ($\text{part}(R_{\text{req}}) \in \mathcal{F}(T_i)$) at time t (summarized in Algorithm 1):

1) **Reentrancy:** If $R_{\text{req}} \in \text{Held}(A_i,t)$, access is granted immediately.
2) **Ceiling Admissibility & Guarded Inheritance Pass-Through Rule:** If R_{req} is currently free, A_i is granted

the lock **if and only if**:

$$\Pi_i^{\text{eff}}(t) > C^*_{-A_i}(t;\mathcal{F}(T_i)) \vee \Big(\text{Held}(A_i,t) \neq \emptyset \ \wedge \ \Pi_i^{\text{eff}}(t) = C^*(t;\mathcal{F}(T_i))\Big) \tag{12}$$

When $\text{Held}(A_i,t) = \emptyset$ (initial admission), the second disjunct is false and A_i must satisfy the strict inequality $\Pi_i^{\text{eff}}(t) > C^*_{-A_i}(t;\mathcal{F}(T_i))$, which holds if and only if $\mathcal{K}_{\text{ext}}(A_i,t) = \emptyset$ and $\mathcal{W}_{\text{pend}}(A_i,t) = \emptyset$. Conversely, once $\text{Held}(A_i,t) \neq \emptyset$, Eq. (11) guarantees that $C^*_{-A_i}(t;\mathcal{F}(T_i)) = 0 < \Pi_i^{\text{eff}}(t)$ (satisfying the first disjunct directly) and $\Pi_i^{\text{eff}}(t) = C^*(t;\mathcal{F}(T_i))$ (satisfying the second disjunct), permitting A_i to acquire all remaining Step-2 locks in $\mathcal{F}(T_i)$ without self-blocking.

3) **Unified Contention & Ceiling Suspension:** If R_{req} is held by $A_{\text{hold}} \neq A_i$ or Eq. (12) fails (both tested in Rule 2), let $\mathcal{B}_{\text{ceil}}(A_i,t) \triangleq \{A_j \in \mathcal{K}_{\text{ext}}(A_i,t) \mid \max_{P_m \in \mathcal{F}(T_i)\cap\mathcal{F}(T_j)} \Pi(P_m,t) \geq \Pi_i^{\text{eff}}(t)\} \cup \{A_j \in \mathcal{K}^{\text{queue}}_{\text{ext}}(A_i,t)\}$ denote the set of external ceiling-blocking lock holders. On each wait iteration, A_i first clears any stale outgoing edges $\mathcal{E}(A_i) \leftarrow \emptyset$ (recomputing Eqs. (13)–(14) for any former blockers), binds directed wait edges $(A_i \to A_j)$ in $\mathcal{G}_{\text{wait}}(t)$ for all $A_j \in (\{A_{\text{hold}}\} \cup \mathcal{B}_{\text{ceil}}(A_i,t))$ (so every intersecting lock holder is elevated simultaneously), propagates $\Pi_i^{\text{eff}}(t)$ and $\tilde{\Pi}_i^{\text{eff}}(t)$ transitively via Eqs. (13)–(14), and suspends on its per-agent condition variable $\text{CV}(A_i)$ with timeout $\Delta t_{\text{wait}} = \infty$ if promoted to Π_{sat} else $\max(0, T_{\text{sat}} - (t - t_{\text{start},i}))$ (so an already-promoted waiter awaits explicit release notification without zero-timeout busy-spinning).

D. Unified Transitive Epistemic Priority Inheritance and Complexity Bound

To prevent unbounded priority inversion under both direct contention and ceiling preemption, let $\text{Waiters}(A_j,t) \triangleq \{A_k \in \mathcal{A} \mid (A_k \to A_j) \in \mathcal{E}(t)\}$ denote all agents blocked by resources held by A_j. The effective preemption priority $\Pi_j^{\text{eff}}(t)$ and effective dynamic urgency $\tilde{\Pi}_j^{\text{eff}}(t)$ of A_j are maintained atomically:

$$\Pi_j^{\text{eff}}(t) \triangleq \max\left(\Pi_j^{\max}, \max_{A_k \in \text{Waiters}(A_j,t)} \Pi_k^{\text{eff}}(t)\right), \tag{13}$$

$$\tilde{\Pi}_j^{\text{eff}}(t) \triangleq \max\left(\tilde{\Pi}_j(t), \max_{A_k \in \text{Waiters}(A_j,t)} \tilde{\Pi}_k^{\text{eff}}(t)\right) \tag{14}$$

Whenever A_i blocks on $A_j \in (\{\text{holder}(R_{\text{req}})\} \cup \mathcal{B}_{\text{ceil}}(A_i,t))$, A_j immediately inherits both $\Pi_i^{\text{eff}}(t)$ and $\tilde{\Pi}_i^{\text{eff}}(t)$. Upon transaction commit (or clean `Biz-Rule Reject` exit), A_j atomically releases all locks in $\text{Held}(A_j,t)$, removes itself from $\mathcal{A}_{\text{active}}(t)$—deleting A_j from $\mathcal{U}(P_m,t)$ and updating each partition max-heap $\Pi(P_m,t)$ for $P_m \in \mathcal{F}(T_j)$—removes all incoming edges $(A_k \to A_j) \in \mathcal{E}(t)$ incident to A_j, rebinds any multi-partition waiter A_k to remaining active lock holders intersecting $\mathcal{F}(T_k)$, refreshes age-dependent waiter scores $\tilde{\mathbf{\Pi}}_k(t)$ and any pending T_{sat} promotions (propagating along remaining wait edges), and signals waiting threads on $\text{CV}(A_k)$ in descending order of live dynamic urgency $\tilde{\mathbf{\Pi}}_k^{\text{eff}}(t)$. Crucially, because Eq. (11) guarantees that active lock holders ($\text{Held}(A_j,t) \neq \emptyset$) have pairwise disjoint footprints and never block on secondary locks within $\mathcal{F}(T_j)$, $\mathcal{G}_{\text{wait}}(t)$ is strictly a **bipartite DAG of maximum path length** 1 (sleeping waiters with $\text{Held} = \emptyset$ and in-degree 0 pointing directly to active lock holders with $\text{Held} \neq \emptyset$ and out-degree 0). Consequently, priority propagation at Line 22 traverses at most $|\mathcal{B}_{\text{ceil}}(A_i,t)| \leq \min(B_{\text{part}}, K-1) < K$ single-hop edges in $\mathcal{O}(K)$ time

Algorithm 1 EPCP Atomic Lock Acquisition & Release

1: **Procedure** ACQUIRELOCK(A_i, R_{req}) **under Coordinator Mutex:**
2: **if** $A_i \notin \mathcal{A}_{\text{active}}(t)$ **then**
3: Insert A_i into $\mathcal{A}_{\text{active}}(t)$ and $\mathcal{U}(P_m,t)$ ($\forall P_m \in \mathcal{F}(T_i)$); update $\Pi(P_m,t)$
4: **end if**
5: **if** part(R_{req}) $\notin \mathcal{F}(T_i)$ **then**
6: Discard local staged writes; memoize Infer(ϕ) by SHA-256 `state_version_digest`; release $\text{Held}(A_i,t)$, clear incoming $(A_k \to A_i) \in \mathcal{E}(t)$, rebind multi-partition waiters to remaining active holders, reset $(\Pi_i^{\text{eff}}(t), \tilde{\Pi}_i^{\text{eff}}(t)) \leftarrow (\Pi_i^{\max}, \tilde{\Pi}_i(t))$, refresh age-dependent $\tilde{\mathbf{\Pi}}_k(t)$ and T_{sat} promotions, and wake waiters in descending order of $\tilde{\mathbf{\Pi}}_k^{\text{eff}}(t)$ while keeping $A_i \in \mathcal{A}_{\text{active}}(t)$; expand $\mathcal{F}(T_i) \leftarrow \mathcal{F}(T_i) \cup \{\text{part}(R_{\text{req}})\}$ preserving $(t_{\text{start},i}, \text{rank}(T_i))$; insert $A_i \in \mathcal{U}(\text{part}(R_{\text{req}}),t)$ and update $\Pi(\text{part}(R_{\text{req}}),t)$;
7: **return** RESTART
8: **end if**
9: **loop**
10: **if** $t - t_{\text{start},i} \geq T_{\text{sat}}$ **then**
11: Promote $\tilde{\mathbf{\Pi}}_i(t), \mathbf{\Pi}_i^{\max} \leftarrow (\lfloor 10^6 \Pi_{\text{sat}} \rfloor, -\text{rank}(T_i))$; update $\Pi(P_m,t)$ and propagate $\Pi_i^{\text{eff}}(t), \tilde{\Pi}_i^{\text{eff}}(t)$ along $\mathcal{G}_{\text{wait}}(t)$ via Eqs. (13)–(14)
12: **end if**
13: **if** $R_{\text{req}} \in \text{Held}(A_i,t)$ **then**
14: **return true**
15: **end if**
16: Compute $C^*_{-A_i}(t;\mathcal{F}(T_i))$ and $C^*(t;\mathcal{F}(T_i))$ via 64-bit footprint masks
17: **if** R_{req} is free **and** Eq. (12) holds **then**
18: $\text{Held}(A_i,t) \leftarrow \text{Held}(A_i,t) \cup \{R_{\text{req}}\}$; clear outgoing $\mathcal{E}(A_i)$ and recompute former blockers;
19: **return true**
20: **end if**
21: Clear stale outgoing $\mathcal{E}(A_i) \leftarrow \emptyset$ (recomputing Eqs. (13)–(14) for former blockers); bind $(A_i \to A_j) \in \mathcal{E}(t)$ for all $A_j \in (\{\text{holder}(R_{\text{req}})\} \cup \mathcal{B}_{\text{ceil}}(A_i,t))$
22: Propagate $\Pi_i^{\text{eff}}(t)$ and $\tilde{\Pi}_i^{\text{eff}}(t)$ transitively along $\mathcal{G}_{\text{wait}}(t)$ via Eqs. (13)–(14)
23: WAIT$\big(\text{CV}(A_i),\ \Delta t_{\text{wait}} = \infty$ if promoted to Π_{sat} else $\max(0, T_{\text{sat}} - (t - t_{\text{start},i}))\big)$
24: **end loop**
25: **Procedure** RELEASEALLLOCKS(A_j) **at Commit / Biz-Rule Exit:**
26: Flush staged writes (if committing); release $\text{Held}(A_j,t)$; remove A_j from $\mathcal{A}_{\text{active}}(t)$, clear incoming edges $(A_k \to A_j) \in \mathcal{E}(t)$ incident to A_j, rebind sleeping multi-partition waiters to any remaining active blockers, and update $\Pi(P_m,t)$; refresh age-dependent $\tilde{\mathbf{\Pi}}_k(t)$ and pending T_{sat} promotions across waiting candidates (propagating along remaining edges in $\mathcal{G}_{\text{wait}}(t)$); wake waiters on $\text{CV}(A_k)$ in descending order of $\tilde{\mathbf{\Pi}}_k^{\text{eff}}(t)$.

(and waking waiters re-evaluate ceilings in $\mathcal{O}(K)$ time via cached `uint64_t` waiter footprint masks), bounding each lock acquisition to $\mathcal{O}(|\mathcal{F}(T_i)| \log K + K)$ time and each unlock with sorted waiter wake-up to $\mathcal{O}(|\mathcal{F}(T_i)| \log K + K \log K)$ time (< 1.8 μs at $K = 40, |\mathcal{F}(T_i)| \leq 3$).

IV. Formal Theoretical Analysis

A. Theorem 1: Deadlock Elimination via Inductive Dynamic Acyclicity

Theorem 1 (Deadlock Elimination)**.** *Under the Epistemic Priority Ceiling Protocol (EPCP), the Unified Wait-For-Graph $\mathcal{G}_{wait}(t)$ is a directed acyclic graph (DAG) for all times $t \geq 0$. Consequently, deadlocks are provably impossible, and the transaction abort rate due to deadlock is identically 0.00%.*

Proof. We prove Theorem 1 by contradiction using mathematical induction over the discrete sequence of lock acquisition and suspension events $0 = t_0 < t_1 < t_2 < \cdots < t_N$. Throughout this section, we write $\Pi_q^{\max}(t)$ for A_q's instantaneous upper-envelope preemption level at time t (equal to its initial $\Pi_q^{\max}$ for $t - t_{\text{start},q} < T_{\text{sat}}$ and promoted to $(\lfloor 10^6 \Pi_{\text{sat}} \rfloor, -\text{rank}(T_q))$ for $t - t_{\text{start},q} \geq T_{\text{sat}}$).

Base Case ($t = 0$): At system initialization, no locks are held and $\mathcal{E}(0) = \emptyset$, which is trivially acyclic.

Inductive Hypothesis: Assume that for all $t < t_k$, $\mathcal{G}_{\text{wait}}(t)$ is a directed acyclic graph (DAG).

Inductive Step ($t = t_k$): Suppose for the sake of contradiction that at event time t_k, a lock request by some agent closes a minimal directed cycle of length $L \geq 2$ in $\mathcal{G}_{\text{wait}}(t_k)$:

$$A_{p_1} \to A_{p_2} \to \cdots \to A_{p_L} \to A_{p_1} \tag{15}$$

where $\text{succ}(m) \triangleq (m \bmod L) + 1$. By Definition 1, every directed edge $(A_{p_m} \to A_{p_{\text{succ}(m)}}) \in \mathcal{E}(t_k)$ requires that $A_{p_{\text{succ}(m)}}$ is currently holding at least one lock ($\text{Held}(A_{p_{\text{succ}(m)}}, t_k) \neq \emptyset$). Since every cycle vertex A_{p_m} has an incoming edge from $A_{p_{(m-2 \bmod L)+1}}$, every A_{p_m} satisfies $\text{Held}(A_{p_m}, t_k) \neq \emptyset$. Because $\mathcal{K}_{\text{ext}}^{\text{queue}}(A_i, t)$ is gated by $\text{Held}(A_i, t) = \emptyset$, $\text{Held}(A_{p_m}, t_k) \neq \emptyset$ immediately implies $\mathcal{K}_{\text{ext}}^{\text{queue}}(A_{p_m}, t_k) = \emptyset$, so any edge $(A_{p_m} \to A_{p_{\text{succ}(m)}}) \in \mathcal{E}(t_k)$ must arise from $\mathcal{K}_{\text{ext}}^{\text{hold}}(A_{p_m}, t_k)$ (or direct row contention within declared footprints) and therefore requires direct footprint intersection:

$$\mathcal{F}(T_{p_m}) \cap \mathcal{F}(T_{p_{\text{succ}(m)}}) \neq \emptyset, \quad \forall m \in \{1, \ldots, L\} \tag{16}$$

(Note that Eq. (11) already implies that active lock holders have pairwise disjoint footprints and out-degree 0; the derivation below proves from first principles why no active lock holder could ever have entered an intersecting footprint to form Eq. (16).)

For each agent A_{p_m} in the cycle, since $\text{Held}(A_{p_m}, t_k) \neq \emptyset$, let $t_{p_m}^{\text{entry}} < t_k$ denote the *most recent* instant prior to t_k when A_{p_m} transitioned from holding no locks ($\text{Held}(A_{p_m}, t_{p_m}^{\text{entry}-}) = \emptyset$) to acquiring the first of its currently held locks $R_{(p_m)}^{\text{first}}$ (guaranteeing that $\text{Held}(A_{p_m}, t) \neq \emptyset$ held continuously throughout $[t_{p_m}^{\text{entry}}, t_k]$ without any intervening lock release or footprint expansion). Because events are serialized by the coordinator mutex, entry timestamps are distinct; let A_{p^*} ($p^* \in \{1, \ldots, L\}$) be the unique agent in the cycle that entered its current critical section latest in time:

$$t^* \triangleq t_{p^*}^{\text{entry}} = \max_{m \in \{1,\ldots,L\}} t_{p_m}^{\text{entry}} < t_k \tag{17}$$

Consider the system state at t^{*-} immediately prior to A_{p^*} being granted $R_{(p^*)}^{\text{first}}$. Because $t_{p_m}^{\text{entry}} < t^*$ for every other cycle member $m \neq p^*$, A_{p^*}'s cycle successor $A_{p_{\text{succ}(p^*)}}$ ($\text{succ}(p^*) \neq p^*$ since $L \geq 2$) had already entered its critical section and was actively holding at least $R_{(p_{\text{succ}(p^*)})}^{\text{first}}$ at t^{*-}, so $\text{Held}(A_{p_{\text{succ}(p^*)}}, t^{*-}) \neq \emptyset$.

By Eq. (16), there exists at least one shared partition $P_{\text{shared}} \in \mathcal{F}(T_{p^*}) \cap \mathcal{F}(T_{p_{\text{succ}(p^*)}})$. By the atomic registration invariant (Section III-B), both A_{p^*} and $A_{p_{\text{succ}(p^*)}}$ belonged to $\mathcal{U}(P_{\text{shared}}, t^{*-})$ at t^{*-}. Furthermore, because Lines 10–12 of Algorithm 1 evaluate any pending T_{sat} promotion and update both $\Pi_{p^*}^{\max}(t^{*-})$ and $\Pi(P_m, t^{*-})$ under the coordinator mutex *prior* to evaluating Eq. (12) at Line 17, the partition priority ceiling of P_{shared} at t^{*-} satisfied:

$$\begin{aligned} \Pi(P_{\text{shared}}, t^{*-}) &= \max_{A_q \in \mathcal{U}(P_{\text{shared}}, t^{*-})} \Pi_q^{\max}(t^{*-}) \\ &\geq \max\left(\Pi_{p^*}^{\max}(t^{*-}),\, \Pi_{p_{\text{succ}(p^*)}}^{\max}(t^{*-})\right) \\ &\geq \Pi_{p^*}^{\max}(t^{*-}) \end{aligned} \tag{18}$$

Because $A_{p_{\text{succ}(p^*)}} \neq A_{p^*}$ satisfied $\text{Held}(A_{p_{\text{succ}(p^*)}}, t^{*-}) \neq \emptyset$ and $P_{\text{shared}} \in \mathcal{F}(T_{p^*}) \cap \mathcal{F}(T_{p_{\text{succ}(p^*)}})$, $A_{p_{\text{succ}(p^*)}} \in \mathcal{K}_{\text{ext}}^{\text{hold}}(A_{p^*}, t^{*-}) \subseteq \mathcal{K}_{\text{ext}}(A_{p^*}, t^{*-})$. By the definition of the External Intersecting-Footprint System Ceiling (Eq. (9)), the external ceiling confronting A_{p^*} at t^{*-} satisfied:

$$C_{-A_{p^*}}^*(t^{*-}; \mathcal{F}(T_{p^*})) \geq \Pi(P_{\text{shared}}, t^{*-}) \geq \Pi_{p^*}^{\max}(t^{*-}) \tag{19}$$

Simultaneously, because A_{p^*} held no locks at t^{*-} ($\text{Held}(A_{p^*}, t^{*-}) = \emptyset$), it had zero incoming wait edges ($\text{Waiters}(A_{p^*}, t^{*-}) = \emptyset$), so its effective priority equaled its instantaneous upper-envelope preemption level: $\Pi_{p^*}^{\text{eff}}(t^{*-}) = \Pi_{p^*}^{\max}(t^{*-})$. Moreover, because $\text{Held}(A_{p^*}, t^{*-}) = \emptyset$, the second disjunct of the EPCP Ceiling Admissibility Rule (Eq. (12)) evaluated to false at t^{*-}. Therefore, A_{p^*} could be granted $R_{(p^*)}^{\text{first}}$ at t^* if and only if the first disjunct of Eq. (12) held strictly:

$$\Pi_{p^*}^{\max}(t^{*-}) = \Pi_{p^*}^{\text{eff}}(t^{*-}) > C_{-A_{p^*}}^*(t^{*-}; \mathcal{F}(T_{p^*})) \tag{20}$$

Combining Eq. (19) and Eq. (20) yields $\Pi_{p^*}^{\max}(t^{*-}) > C_{-A_{p^*}}^*(t^{*-}; \mathcal{F}(T_{p^*})) \geq \Pi_{p^*}^{\max}(t^{*-})$, i.e., $\Pi_{p^*}^{\max}(t^{*-}) > \Pi_{p^*}^{\max}(t^{*-})$, which is a direct mathematical contradiction. Thus, no cycle can ever form, and $\mathcal{G}_{\text{wait}}(t)$ remains a strict DAG for all $t \geq 0$. □

B. Theorem 2: Bounded Priority Inversion

Theorem 2 (Bounded Priority Inversion)**.** *Under EPCP, a high-priority agent A_H can be blocked by lower-preemption-level agents ($\Pi_L^{\max} < \Pi_H^{\max}$ with lower live urgency $\tilde{\Pi}_L^{\text{eff}}(t) < \tilde{\Pi}_H^{\text{eff}}(t)$) for at most the duration of a single critical section per mutually disjoint partition component in $\mathcal{F}(T_H)$ (at most one lower-priority lock holder across all mutually overlapping*

TABLE I

REGIME 1 ENTERPRISE CONCURRENCY TELEMETRY: STRICT 2PL VS. EPCP UNDER ZIPFIAN CONTENTION ($\theta = 0.85$, FRESHLY RE-SEEDED PER RUN, 5 SEEDS MEAN ± SD; TOTAL TX = COMMITTED + BIZ-RULE REJECT + DEADLOCKS)

Protocol	K	Total Tx	Committed	Biz-Rule Reject†	Deadlocks	Abort (%)	Duration (s)	Goodput (TPS)‡	Total TPS‡	All Mean Lat (ms)	Comm. Mean Lat (ms)§	All / Comm. P95 Lat (ms)§	All P99 / Max Lat (ms)¶
S2PL	5	125	125.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	$0.00 \pm 0.00\%$	0.137 ± 0.006	913.21 ± 38.4	913.21 ± 38.4	4.50 ± 0.21	4.50 ± 0.21	19.03 ± 1.12 / 19.03	41.97 / 48.60
EPCP	5	125	125.0 ± 0.0	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	0.116 ± 0.005	$\mathbf{1{,}075.45 \pm 42.1}$	$\mathbf{1{,}075.45 \pm 42.1}$	$\mathbf{2.80 \pm 0.14}$	$\mathbf{2.80 \pm 0.14}$	$\mathbf{3.09 \pm 0.18}$ / $\mathbf{3.09}$	65.37 / 74.20
S2PL	10	250	250.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	$0.00 \pm 0.00\%$	0.307 ± 0.014	814.94 ± 35.2	814.94 ± 35.2	9.77 ± 0.45	9.77 ± 0.45	46.25 ± 2.45 / 46.25	64.49 / 78.90
EPCP	10	250	250.0 ± 0.0	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	0.296 ± 0.012	$\mathbf{845.69 \pm 33.8}$	$\mathbf{845.69 \pm 33.8}$	$\mathbf{7.36 \pm 0.38}$	$\mathbf{7.36 \pm 0.38}$	$\mathbf{3.23 \pm 0.19}$ / $\mathbf{3.23}$	225.20 / 258.40
S2PL	20	500	468.0 ± 5.2	29.0 ± 4.8	3.0 ± 1.0	$0.60 \pm 0.20\%$	0.797 ± 0.035	586.84 ± 25.1	627.35 ± 26.8	27.44 ± 1.62	28.95 ± 1.70	114.72 ± 6.30 / 118.40	194.80 / 234.50
EPCP	20	500	413.0 ± 6.1	87.0 ± 6.1	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	0.693 ± 0.029	$\mathbf{596.06 \pm 24.3}$	$\mathbf{721.50 \pm 29.4}$	$\mathbf{17.40 \pm 1.05}$	$\mathbf{20.62 \pm 1.24}$	$\mathbf{3.24 \pm 0.21}$ / $\mathbf{3.41}$	580.76 / 642.40
S2PL	30	750	505.0 ± 8.4	241.0 ± 8.1	4.0 ± 1.3	$0.53 \pm 0.18\%$	1.482 ± 0.068	340.70 ± 15.6	506.07 ± 23.1	52.76 ± 3.10	71.40 ± 4.15	238.66 ± 12.80 / 264.10	304.37 / 418.00
EPCP	30	750	444.0 ± 7.9	306.0 ± 7.9	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	1.667 ± 0.071	266.41 ± 11.4	449.91 ± 19.2	$\mathbf{43.16 \pm 2.45}$	$\mathbf{64.85 \pm 3.60}$	429.58 ± 21.50 / 462.30	818.22 / 912.50
S2PL	40	1000	376.0 ± 9.5	620.0 ± 9.2	4.0 ± 1.4	$0.40 \pm 0.14\%$	2.537 ± 0.112	148.18 ± 6.5	394.17 ± 17.4	93.65 ± 5.40	182.10 ± 9.80	302.18 ± 16.40 / 348.50	370.22 / 582.10
EPCP	40	1000	318.0 ± 8.8	682.0 ± 8.8	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	1.937 ± 0.084	$\mathbf{164.21 \pm 7.1}$	$\mathbf{516.26 \pm 22.3}$	$\mathbf{47.13 \pm 2.80}$	$\mathbf{108.40 \pm 5.90}$	494.36 ± 24.80 / 538.20	801.65 / 924.80
Zero-Depletion Controls Across $K \in \{20, 30, 40\}$ *(100× Initial Balance/Stock Snapshot; 0* `Biz-Rule Rejects`*, Isolating Pure Concurrency Arbitration on Full Workflows)*													
S2PL (No-Depl.)	20	500	495.0 ± 1.4	0.0 ± 0.0	5.0 ± 1.4	$1.00 \pm 0.28\%$	0.884 ± 0.038	559.95 ± 24.1	565.61 ± 24.3	31.20 ± 1.75	31.40 ± 1.78	124.50 ± 6.80 / 125.10	204.60 / 248.00
EPCP (No-Depl.)	20	500	$\mathbf{500.0 \pm 0.0}$	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	$\mathbf{0.742 \pm 0.031}$	$\mathbf{673.85 \pm 28.2}$	$\mathbf{673.85 \pm 28.2}$	$\mathbf{21.10 \pm 1.15}$	$\mathbf{21.10 \pm 1.15}$	$\mathbf{3.45 \pm 0.22}$ / $\mathbf{3.45}$	612.50 / 684.20
S2PL (No-Depl.)	30	750	742.0 ± 1.8	0.0 ± 0.0	8.0 ± 1.8	$1.07 \pm 0.24\%$	2.105 ± 0.092	352.49 ± 15.4	356.29 ± 15.6	78.40 ± 4.20	78.95 ± 4.25	272.10 ± 14.10 / 273.40	385.20 / 468.00
EPCP (No-Depl.)	30	750	$\mathbf{750.0 \pm 0.0}$	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	$\mathbf{1.790 \pm 0.076}$	$\mathbf{418.99 \pm 17.8}$	$\mathbf{418.99 \pm 17.8}$	$\mathbf{54.20 \pm 2.95}$	$\mathbf{54.20 \pm 2.95}$	318.40 ± 16.20 / 318.40	742.10 / 846.00
S2PL (No-Depl.)	40	1000	989.0 ± 2.2	0.0 ± 0.0	11.0 ± 2.2	$1.10 \pm 0.22\%$	4.120 ± 0.185	240.05 ± 10.8	242.72 ± 10.9	142.80 ± 7.90	143.60 ± 7.95	385.40 ± 19.50 / 386.20	524.00 / 648.00
EPCP (No-Depl.)	40	1000	$\mathbf{1{,}000.0 \pm 0.0}$	0.0 ± 0.0	$\mathbf{0.0 \pm 0.0}$	$\mathbf{0.00 \pm 0.00\%}$	$\mathbf{3.390 \pm 0.142}$	$\mathbf{294.99 \pm 12.3}$	$\mathbf{294.99 \pm 12.3}$	$\mathbf{86.40 \pm 4.60}$	$\mathbf{86.40 \pm 4.60}$	412.50 ± 20.80 / 412.50	786.00 / 892.00

†**Biz-Rule Reject:** Clean application-level domain exit immediately upon Step-1 lock acquisition when a hot Zipfian account/SKU ($\theta = 0.85$) has insufficient balance/stock (0 LLM tokens wasted; 0 concurrency rollbacks).

‡**Goodput (TPS)** and **Total TPS** are computed per seed $s \in \{1, \ldots, 5\}$ ($\text{Committed}_s/\text{Duration}_s$ and Total $\text{Tx}_s/\text{Duration}_s$) and reported as 5-seed Mean ± SD. Under S2PL, deadlock victims are aborted by the WFG detector without internal retry.

§**Comm. Mean / P95 Lat:** Latency restricted to full committed settlements. Because `Biz-Rule Rejects` exit in < 0.2 ms after Step-1 lock acquisition, `All Mean Lat` is lower than `Comm. Mean Lat` when rejects occur ($K \geq 20$).

¶**P99 / Max Lat, Quantile Bound & Tail Draining:** At $K \leq 20$, $\geq 95\%$ of EPCP requests enter unblocked partitions in ≤ 3.45 ms, while the lowest-priority $< 4\%$ waiters queued at $\text{CV}(A_i)$ absorb multi-step queue waiting (580.76–684.20 ms at $K = 20$), satisfying $\mu \leq 0.95P_{95} + 0.04P_{99} + 0.01\text{Max}$ across all rows.

footprints, and at most $|\mathcal{F}(T_H)|$ concurrent lower-priority critical sections across disjoint partitions).

Proof. Suppose agent A_H requests its first lock at time t with priority $\Pi_H^{\max}$ and is suspended under Section III-C by a lower-priority agent A_L ($\Pi_L^{\max} < \Pi_H^{\max}$) whose footprint intersects $\mathcal{F}(T_H)$ at shared partition $P_m \in \mathcal{F}(T_H) \cap \mathcal{F}(T_L)$ with $\Pi(P_m, t) \geq \Pi_H^{\max}$. First, among any set of lower-priority agents whose footprints overlap on P_m (or overlap with one another), at most *one* such lower-priority agent can hold a lock at time t: if two overlapping lower-priority agents A_{L_1} and A_{L_2} ($\mathcal{F}(T_{L_1}) \cap \mathcal{F}(T_{L_2}) \neq \emptyset$) both held locks, the second to enter ($t_{L_2}^{\text{entry}} > t_{L_1}^{\text{entry}}$) had $\text{Held}(A_{L_2}, t_{L_2}^{\text{entry}-}) = \emptyset$ and would have been required by Eq. (12) to satisfy $\Pi_{L_2}^{\max}(t_{L_2}^{\text{entry}-}) > C^*_{-A_{L_2}}(t_{L_2}^{\text{entry}-}; \mathcal{F}(T_{L_2})) \geq \Pi(P_m, t_{L_2}^{\text{entry}-}) \geq \Pi_{L_2}^{\max}(t_{L_2}^{\text{entry}-})$, which is impossible whenever $\mathcal{K}_{\text{ext}}(A_{L_2}, t_{L_2}^{\text{entry}-}) \neq \emptyset$. Across at most $|\mathcal{F}(T_H)|$ mutually disjoint partitions, at most one pre-existing lower-priority holder per disjoint partition can exist at A_H's arrival, and they execute concurrently in parallel. Furthermore, if $|\mathcal{F}(T_H)| \geq 2$ and one disjoint blocker A_{L_1} on $P_1 \in \mathcal{F}(T_H)$ commits while A_{L_2} on $P_2 \in \mathcal{F}(T_H)$ is still executing, RELEASEALLLOCKS(A_{L_1}) keeps A_H bound to A_{L_2} ($(A_H \to A_{L_2}) \in \mathcal{E}(t)$) and registers $A_H \in \mathcal{W}_{\text{pend}}(A_{L_3}, t)$ until A_H acquires its first lock; by $\mathcal{K}_{\text{ext}}^{\text{queue}}(A_{L_3}, t)$ and $\mathcal{W}_{\text{pend}}(A_{L_3}, t)$ in Eq. (9), any newly arriving or woken lower-priority agent A_{L_3} on P_1 ($\mathbf{\Pi}_{L_3}^{\text{eff}}(t) < \mathbf{\Pi}_H^{\text{eff}}(t)$ and $\tilde{\mathbf{\Pi}}_{L_3}^{\text{eff}}(t) < \tilde{\mathbf{\Pi}}_H^{\text{eff}}(t)$) confronts $C^*_{-A_{L_3}}(t; \mathcal{F}(T_{L_3})) \geq \Pi(P_1, t) \geq \Pi_H^{\max} > \Pi_{L_3}^{\text{eff}}(t)$ both while A_H awaits P_2 and during the Mesa condition-variable handoff window after A_{L_2} commits before A_H's thread re-acquires the coordinator mutex.

Second, by Unified Transitive Priority Inheritance (Eqs. (13)–(14)), each blocking holder A_L immediately inherits $\Pi_L^{\text{eff}}(t) \geq \Pi_H^{\max}$ and $\tilde{\Pi}_L^{\text{eff}}(t) \geq \tilde{\Pi}_H^{\text{eff}}(t)$. Because $\text{Held}(A_L, t) \neq \emptyset$, single-holder admission exclusion across intersecting footprints (Eq. (11)) guarantees that $\mathcal{K}_{\text{ext}}(A_L, t) = \emptyset$, so $C^*_{-A_L}(t; \mathcal{F}(T_L)) = 0 < \Pi_L^{\text{eff}}(t)$ (and simultaneously $\Pi_L^{\text{eff}}(t) = C^*(t; \mathcal{F}(T_L))$ under Eq. (10)). Consequently, the first disjunct (as well as the second disjunct) of Eq. (12) allows A_L to acquire any remaining Step-2 locks in $\mathcal{F}(T_L)$ without secondary ceiling self-blocking, complete its single critical section, and wake waiting threads in descending order of live dynamic urgency $\tilde{\mathbf{\Pi}}_k^{\text{eff}}(t)$ upon commit (where if another waiter $A_{L'}$ with lower static depth $d_{L'} < d_H$ is dispatched ahead of an unpromoted A_H because $\tilde{\mathbf{\Pi}}_{L'}^{\text{eff}}(t) > \tilde{\mathbf{\Pi}}_H^{\text{eff}}(t)$ due to higher accumulated token/aging investment $\Omega_{L'}(t)$, $\mathcal{W}_{\text{pend}}$ permits $A_{L'}$'s admission and that dispatch reflects intentional epistemic sunk-cost scheduling under $\tilde{\Pi}^{\text{eff}}(t)$ rather than priority inversion). □

C. Theorem 3: Conflict Serializability and Starvation-Free Progress

Theorem 3 (Serializability and Starvation Freedom)**.** *Every execution schedule permitted under EPCP is conflict serializable, and every active transaction terminates in finite time without deadlock or starvation.*

Proof. First, consider conflict serializability. If a transaction T_i ever requests an out-of-footprint resource $\text{part}(R_{\text{req}}) \notin \mathcal{F}(T_i)$ at Line 5 of Algorithm 1, its pre-expansion pass discards its uncommitted local write buffer with zero physical database mutations, releases all held locks, expands $\mathcal{F}(T_i) \leftarrow \mathcal{F}(T_i) \cup \{\text{part}(R_{\text{req}})\}$, and returns RESTART to re-execute from Step 1 ($\text{Held}(A_i, t) = \emptyset$). Since $|\mathcal{F}(T_i)| \in \{1, \ldots, B_{\text{part}}\}$ strictly increases on each expansion, at most $B_{\text{part}} - 1 < 64$ transient RESTART transitions can occur. In the final committed pass, all locks are acquired during the growth phase and held until transaction commit or domain validation exit (Strict 2PL); moreover, if Step 1 replays a memoized $\text{Infer}_1(\phi)$ output from the local context cache, the SHA-256 `state_version_digest` check is evaluated *after* R_a is locked in the final pass, guaranteeing that R_a's read attributes are bit-for-bit identical to the state on which $\text{Infer}_1(\phi)$ was computed. By Eswaran et al. [3], any schedule obeying strict two-phase locking is conflict serializable.

Second, by Theorem 1, $\mathcal{G}_{\text{wait}}(t)$ is a strict DAG (0 deadlock aborts). To prove unconditional starvation freedom even under an unbounded continuous arrival stream of deeper ($d_j > d_i$) transactions where $\mathcal{A}_{\text{active}}(t)$ never empties, observe first that across the at most $B_{\text{part}} - 1 < 64$ transient RESTART transitions, A_i remains in $\mathcal{A}_{\text{active}}(t)$ with preserved $(t_{\text{start},i}, \text{rank}(T_i))$ and immediately re-invokes ACQUIRELOCK

in bounded CPU time. In any pass where T_i waits, Algorithm 1 (Lines 10–12, 23, and 25–26) guarantees that T_i either wakes via its bounded timeout at $t_{\text{start},i} + T_{\text{sat}}$ or is refreshed during RELEASEALLLOCKS, atomically promoting *both* $\tilde{\mathbf{\Pi}}_i(t)$ and $\mathbf{\Pi}_i^{\max}$ to the exact 64-bit integer lexicographical pair $\big(\lfloor 10^6 \Pi_{\text{sat}} \rfloor, -\text{rank}(T_i)\big)$, recomputing $\Pi(P_m, t)$ for all $P_m \in \mathcal{F}(T_i)$, and propagating the promoted priority along $\mathcal{G}_{\text{wait}}(t)$ via Eqs. (13)–(14). Because the coordinator compares exact 64-bit integer tuples $\big(\lfloor 10^6 \Pi_{\text{sat}} \rfloor, -\text{rank}(T_i)\big) \in \mathbb{Z}_{\geq 0} \times \mathbb{Z}_{<0}$ and arrival ranks are strictly increasing with arrival time, $-\text{rank}(T_i) > -\text{rank}(T_j)$ holds for every transaction T_j arriving after T_i ($\text{rank}(T_i) < \text{rank}(T_j)$). Furthermore, by upward induction on $\text{rank}(T_q)$ starting from the oldest active transaction $T_{\min} = \arg\min_{T_q \in \mathcal{A}_{\text{active}}(t)} \text{rank}(T_q)$: any active lock holder A_k currently holding a partition in $\mathcal{F}(T_{\min})$ has $\text{Held}(A_k, t) \neq \emptyset$ and by Eq. (11) acquires its remaining in-footprint locks without secondary ceiling blocking and finishes its critical section in bounded LLM/SQL steps; once $T_{\min}$ reaches $t_{\text{start},\min} + T_{\text{sat}}$, its promoted tuple $(\lfloor 10^6 \Pi_{\text{sat}} \rfloor, -\text{rank}(T_{\min}))$ strictly dominates every other active or newly arriving transaction under both $\mathbf{\Pi}^{\text{eff}}$ and $\tilde{\mathbf{\Pi}}^{\text{eff}}$, and $\mathcal{K}_{\text{ext}}^{\text{queue}}$ together with $\mathcal{W}_{\text{pend}}$ in Eq. (9) prevents any transaction T_j ($\text{rank}(T_j) > \text{rank}(T_{\min})$) from entering any freed partition $P_m \in \mathcal{F}(T_{\min})$ ahead of $T_{\min}$ while $T_{\min}$ awaits its remaining partitions or wakes from $\text{CV}(A_{\min})$. Thus $T_{\min}$ acquires all locks and commits in finite time, inductively advancing every subsequent transaction T_i to completion in finite time. □

V. EMPIRICAL EVALUATION AND BENCHMARK RESULTS

A. Experimental Architecture and Methodology

We evaluated EPCP across two primary benchmark regimes plus a 3-table footprint-expansion sensitivity study on an Apple Silicon M-series workstation (8 CPU cores, APFS storage) fronting a local relational SQLite WAL storage engine (`PRAGMA journal_mode = WAL; PRAGMA synchronous = NORMAL; PRAGMA busy_timeout = 60000;`). To prevent SQLite's file-level single-writer lock from artificially serializing multi-step reasoning phases while guaranteeing zero dirty or stale reads, the prototype coordinator maintains a synchronized in-memory mirror of committed row state alongside the persistent SQLite WAL audit ledger: all row- and partition-level lock arbitration, condition-variable suspension ($\text{CV}(A_i)$), Step-1 `Biz-Rule Reject` checks, and intermediate reasoning steps ($\text{Infer}(\phi)$) read the exact latest committed state under held locks and stage mutations in a transaction-local write buffer; once all locks in T_i are held and validated, the final physical SQLite mutation batch (`BEGIN IMMEDIATE; UPDATE ...; COMMIT;`) flushes to the WAL journal and updates the committed mirror atomically in $\approx$ 50–150 μs at the Commit step. In Regime 2 (where throughput is ≤ 20 TPS), this 50–150 μs commit flush represents $< 0.3\%$ wall-time utilization, whereas in Regime 1 (where peak goodput reaches 673.85–1,075.45 TPS), global single-writer WAL flush serialization accounts for $\approx 5\%$–16% of wall-clock time equally across all compared protocols (an ablation disabling the physical SQLite WAL flush in Regime 1 increases pure in-memory committed goodput at $K = 20$ from 673.85 TPS to 768.40 TPS for EPCP and from 559.95 TPS to 628.10 TPS for S2PL, widening EPCP's relative goodput advantage from $+20.3\%$ to $+22.3\%$):

1) **Regime 1 (High-Frequency Lock-Manager Stress Test, Table I):** Isolates raw lock-table arbitration and ceiling condition-variable overhead using in-critical-section CPU-bound cryptographic HMAC-SHA256 policy verification and polynomial risk scoring ($\approx$ 0.5–2.0 ms per lock step, $|\mathcal{F}(T_i)| = 2$) across $K \in \{5, 10, 20, 30, 40\}$ concurrent worker threads (25 transactions/thread, 125 to 1,000 transactions/run, repeated across 5 independent random seeds). Prior to each $(K, \text{Protocol})$ run, the database is freshly re-seeded from a canonical snapshot of 10,000 Customer Accounts (initial balances \$2,500–\$15,000) and 5,000 Inventory Items (initial stock 15–45 units/SKU, where each settlement deducts a depth-correlated lot quantity $q_i = 1 + 2d_i \in \{3, 5, 7, 9\}$ units and corresponding settlement value \$150–\$950) under heavy Zipfian contention ($\theta = 0.85$, concentrating 85% of accesses on the top 50 hot accounts and top 30 hot inventory items, supplemented by a Zipfian skew sensitivity sweep across $\theta \in \{0.0, 0.50, 0.70, 0.85, 0.99\}$ in Section V-B). In addition to the standard finite-stock snapshot (where hot SKUs naturally experience stock depletion at $K \geq 20$), we execute **Zero-Depletion Controls across** $K \in \{20, 30, 40\}$ with $100\times$ initial balances/stock so 0 `Biz-Rule Rejects` occur and 100% of non-deadlocked transactions execute full 2-step settlements.
2) **Regime 2 (High-Latency 500 ms–1,500 ms 2-Step LLM Reasoning Benchmark & Live Vertex AI Evaluation, Table II):** Evaluates 2-step agent transactions ($\text{Lock}(R_a) \rightarrow \text{Infer}_1(\phi) \rightarrow \text{Lock}(R_b) \rightarrow \text{Infer}_2(\phi) \rightarrow$ Commit, with 25 transactions/thread $= 500$ attempts at $K = 20$ and 1,000 attempts at $K = 40$ per seed across 5 seeds on the $100\times$ Zero-Depletion snapshot, $|\mathcal{F}(T_i)| = 2$ over $B_{\text{part}} = 64$, $d_i \sim \text{Uniform}\{1, 2, 3, 4\}$, initial $\mathcal{H}_i \sim \text{Beta}(2, 5)$, 800 non-preemptible LLM tokens billed per Infer step $=$ 1,600 tokens per full 2-step attempt, and uniform 250–750 ms reasoning latency per step $=$ 500–1,500 ms total 2-step in-lock reasoning duration per transaction, mean 1.00 s, giving a Little's Law zero-wait upper bound of 20.0 Tx/s at $K = 20$ and 40.0 Tx/s at $K = 40$). Table II reports single-pass attempt metrics alongside cumulative token waste per 1,000 committed workflows under both the stationary geometric projection $\frac{\text{Attempt Waste}}{1 - r_{\text{abort}}}$ and closed-loop timestamp-preserving retry execution (where aborted transactions preserve their initial birth timestamp $t_{\text{start},i}$ across retries as required by Rosenkrantz et al. [5] until all 1,000 workflows commit). We compare 11 protocols and ablations spanning all canonical concurrency control families [13]: **S2PL** [3], **2PL-Wait Promote** [9]

(applying transitive epistemic priority inheritance $\tilde{\Pi}_j^{\text{eff}}(t)$ on direct row contention without EPCP's ceiling admission gate), **Wound-Wait** [5], [6], **Wait-Die** [5], **OCC (Snapshot Read + CAS Commit)** [12], **Coarse Shard-Scoped C2PL (`Shard-C2PL`, $B_{\text{shard}} = 8$)** [2], **Fine-Grained Partition C2PL (`Part-C2PL`, $B_{\text{part}} = 64$, FIFO)** (acquiring all partitions in $\mathcal{F}(T_i)$ atomically upfront with FIFO wait queues), **64-Partition Canonical Ordered Locking (`Part-Canonical`, $B_{\text{part}} = 64$)** (acquiring partition locks sequentially in ascending order $\min(\mathcal{F}(T_i)) < \max(\mathcal{F}(T_i))$ prior to Step 1 without a speculative pre-pass), **Row-Level Canonical Ordered Locking (Discovery Pre-Pass)** (executing an unlocked speculative Infer_1 pre-pass to discover R_b, sorting $\min(R_a, R_b) < \max(R_a, R_b)$, and re-running Infer_1 if R_a mutated prior to lock acquisition), **EPCP Ablation (Static $\Pi_i^{\max}$ Only, w/o Live $\tilde{\Pi}_i(t)$)**, and **Full EPCP**. Finally, we supplement the controlled sweep with a **Live Google Cloud Vertex AI (`gemini-2.5-flash`) Multi-Agent Evaluation** across $K \in \{10, 20\}$ concurrent worker threads (3 independent seeds across S2PL, Wound-Wait, OCC, EPCP Static-$\Pi_i^{\max}$, and Full EPCP on the $100\times$ Zero-Depletion snapshot).

B. Analysis of Empirical Telemetry

1) **Total Elimination of Deadlocks and Token Waste Across All 11 Protocols and Ablations:** Across all concurrency levels in Table I and Table II (and Figure 1), EPCP recorded **0 deadlocks** (0.00% abort rate) and **0 wasted LLM tokens**. In Regime 2 ($K = 40$, Table II), S2PL suffered 18.40% deadlock aborts at Lock 2 (147.2k wasted tokens per 1,000 attempts / 180.4k per 1,000 committed workflows). Notably, `2PL-Wait Promote` [9]—which includes EPCP's epistemic priority inheritance on direct row contention but omits the partition ceiling admission gate—still suffered 16.20% WFG deadlock aborts (129.6k wasted tokens/1k attempts), proving that priority inheritance alone cannot prevent multi-resource cycles without ceiling admission control. Timestamp-based Wound-Wait aborted 31.20% (374.4k wasted tokens/1k attempts; 421.8k/1k committed in closed-loop timestamp-preserving retry runs), Wait-Die aborted 36.50% (178.4k wasted tokens/1k attempts; 218.4k/1k committed in closed-loop retry runs), and OCC [12] failed validation on 64.80% of attempts (1,036.8k wasted tokens/1k attempts; 2,945.5k geometric / 3,112.0k closed-loop per 1k committed). Under closed-loop retry execution at $K = 40$ (where every aborted workflow retries with preserved initial birth timestamp $t_{\text{start},i}$ until all 1,000 workflows commit), effective completion goodput drops to 11.6 Tx/s for S2PL, 9.4 Tx/s for Wound-Wait, 8.1 Tx/s for Wait-Die, and 3.2 Tx/s for OCC, compared to 19.6 Tx/s for Full EPCP. Comparing the five non-aborting partition-aware rows in Table II cleanly isolates partition granularity from epistemic scheduling: moving from Coarse Shard-Scoped C2PL (`Shard-C2PL`, $B_{\text{shard}} = 8$, 9.2 Tx/s) to 64-Partition Canonical Ordered Locking (`Part-Canonical`, $B_{\text{part}} = 64$, 16.6 Tx/s, P95 4.18 s) and Fine-Grained Partition C2PL (`Part-C2PL`, $B_{\text{part}} = 64$ FIFO, 17.4 Tx/s, P95 4.02 s) captures the static 64-partition granularity gain, while adding static depth/rank ceilings (EPCP Static-$\Pi_i^{\max}$, 17.8 Tx/s, P95 3.89 s) and Full EPCP's live dynamic urgency $\tilde{\Pi}_i^{\text{eff}}(t)$ wake-up ordering and transitive priority inheritance (19.6 Tx/s, P95 3.41 s) yields a $+12.6\%$ goodput gain and 15.2% lower committed P95 latency over FIFO `Part-C2PL` on the Zero-Depletion snapshot (and in finite-stock Regime 2 runs at $10\times$ initial stock, $K = 40$, where Step-1 `Biz-Rule Rejects` occur, Full EPCP achieves 23.5 ± 0.7 Tx/s and 2.94 s P95 latency vs. 20.4 ± 0.6 Tx/s and 3.56 s P95 for FIFO `Part-C2PL`, a $+15.2\%$ goodput gain).

2) **Accounting for Application-Level Business-Rule Rejections, Zero-Depletion Controls, and Zipfian Skew Sensitivity:** In Table I, every transaction is accounted for by Total Tx = Committed + Biz-Rule Reject + Deadlocks. Because each settlement deducts a depth-correlated lot quantity $q_i = 1 + 2d_i \in \{3, 5, 7, 9\}$ units from a finite initial stock (15–45 units/SKU) and EPCP dispatches deeper ($d_i \in \{3, 4\}$) high-quantity settlements first, EPCP clears the same physical inventory volume in fewer, larger-lot committed settlements (413 vs. 468 at $K = 20$, 444 vs. 505 at $K = 30$, and 318 vs. 376 at $K = 40$) and triggers fast < 0.2 ms `Biz-Rule Reject` exits earlier on subsequent shallow requests (at $K = 30$, 444 full 2-step settlements still contend across partition queues before $K = 40$'s 682 sub-millisecond rejects dominate total throughput, explaining the single non-bolded finite-stock goodput cell at $K = 30$). In the **Zero-Depletion Controls across** $K \in \{20, 30, 40\}$ (bottom of Table I), where initial balances/stock are scaled $100\times$ so 0 `Biz-Rule Rejects` occur across all K, EPCP commits 100% of transactions ($500/500$, $750/750$, and $1{,}000/1{,}000$) with 0 deadlocks and strictly outperforms S2PL at every concurrency level—achieving $+20.3\%$ higher goodput at $K = 20$ (673.85 vs. 559.95 TPS), $+18.9\%$ higher goodput at $K = 30$ (418.99 vs. 352.49 TPS), and $+22.9\%$ higher goodput at $K = 40$ (294.99 vs. 240.05 TPS), while cutting committed mean latency by 32.8% at $K = 20$ (21.10 vs. 31.40 ms), 31.3% at $K = 30$ (54.20 vs. 78.95 ms), and 39.8% at $K = 40$ (86.40 vs. 143.60 ms), a 31.3%–39.8% reduction across $K \in \{20, 30, 40\}$. Furthermore, a Zipfian skew sensitivity sweep across $\theta \in \{0.0, 0.50, 0.70, 0.85, 0.99\}$ at $K = 20$ (Zero-Depletion) confirms the exact false-sharing boundary predicted in Section II-A: under uniform access ($\theta = 0.0$), row-level S2PL achieves 742.10 TPS vs. EPCP's 718.50 TPS (-3.2% due to the $1/64 \approx 1.56\%$ hash-bucket collision rate across disjoint rows), with the crossover occurring at moderate skew $\theta = 0.50$ (684.20 TPS EPCP vs.

TABLE II

REGIME 2: MULTI-STEP LLM REASONING BENCHMARK (250–750 MS/STEP = 500–1,500 MS TOTAL 2-STEP IN-LOCK REASONING DELAY, 800 TOK/STEP = 1,600 TOK/ATTEMPT, $K \in \{20, 40\}$, 5 SEEDS, MEAN ± SD; ABORT & TOKEN WASTE COLUMNS MEASURED AT $K = 40$)

Protocol / Ablation	**Concurrency Aborts / Re-Inferences at $K = 40$ (%)**	**Wasted Tokens / 1k Attempts ($K = 40$)†**	**Wasted Tokens / 1k Committed ($K = 40$)†**	**Goodput at $K = 20$ (Tx/s)‡**	**Goodput at $K = 40$ (Tx/s)‡**	**Comm. P95 Latency at $K = 40$ (s)**
Strict 2PL (S2PL) [3]	$18.40 \pm 1.85\%$ (WFG Deadlock on Lock 2)	$147.2k \pm 14.8k$	$180.4k \pm 18.1k$	9.4 ± 0.5	14.2 ± 0.9	4.82 ± 0.31
2PL-Wait Promote (Epistemic Inheritance, No Ceiling) [9]	$16.20 \pm 1.65\%$ (WFG Deadlock on Lock 2)	$129.6k \pm 13.2k$	$154.7k \pm 15.8k$	9.9 ± 0.5	15.3 ± 0.8	4.48 ± 0.29
Wound-Wait [5], [6]	$31.20 \pm 2.40\%$ (15.6% Victim in Infer_1 / 15.6% in Infer_2)	$374.4k \pm 28.8k$	$544.2k \pm 41.9k$	8.6 ± 0.5	12.8 ± 0.8	5.45 ± 0.42
Wait-Die [5]	$36.50 \pm 2.65\%$ (14.2% Lock-1 / 22.3% Lock-2 Die)	$178.4k \pm 14.6k$	$280.9k \pm 23.0k$	8.2 ± 0.6	11.9 ± 0.7	5.88 ± 0.45
OCC (Snapshot Read + CAS Commit) [12]	$64.80 \pm 3.10\%$ (Commit CAS Validation Fail)	$1,036.8k \pm 49.0k$	$2,945.5k \pm 139.2k$	6.5 ± 0.4	8.1 ± 0.7	7.90 ± 0.58
Coarse Shard-Scoped C2PL (`Shard-C2PL`, $B_{\text{shard}} = 8$) [2]	$0.00 \pm 0.00\%$ (Upfront Coarse Shard Lock)	$0.0k \pm 0.0k$	$0.0k \pm 0.0k$	5.8 ± 0.3	9.2 ± 0.5	6.74 ± 0.38
64-Partition Canonical Ordered Locking (`Part-Canonical`, $B_{\text{part}} = 64$)	$0.00 \pm 0.00\%$ (Sequential Partition Lock)	$0.0k \pm 0.0k$	$0.0k \pm 0.0k$	10.1 ± 0.4	16.6 ± 0.6	4.18 ± 0.25
Fine-Grained Partition C2PL ($B_{\text{part}} = 64$, FIFO) [2]	$0.00 \pm 0.00\%$ (Upfront 64-Part. FIFO Lock)	$0.0k \pm 0.0k$	$0.0k \pm 0.0k$	10.4 ± 0.4	17.4 ± 0.6	4.02 ± 0.23
Row-Level Canonical Ordered Locking (Discovery Pre-Pass)	$28.40 \pm 2.10\%$ (Pre-Lock Drift Re-Infer; 0.00% Abort)	$227.2k \pm 16.8k$	$227.2k \pm 16.8k$	9.1 ± 0.5	15.1 ± 0.7	4.35 ± 0.28
EPCP Ablation (Static Π_i^{static} Only, w/o Live $\vec{\Pi}_i(t)$)	$0.00 \pm 0.00\%$	$0.0k \pm 0.0k$	$0.0k \pm 0.0k$	10.6 ± 0.4	17.8 ± 0.6	3.89 ± 0.22
EPCP (Full: Static Π_i^{pass} + Live Epistemic $\vec{\Pi}_i(t)$, $B_{\text{part}} = 64$)	$\mathbf{0.00 \pm 0.00\%}$	$\mathbf{0.0k \pm 0.0k}$	$\mathbf{0.0k \pm 0.0k}$	$\mathbf{11.4 \pm 0.4}$	$\mathbf{19.6 \pm 0.6}$	$\mathbf{3.41 \pm 0.19}$

†**Exact Per-Step Token Waste Accounting:** Each 2-step transaction expends 800 tok in Infer_1 and 800 tok in Infer_2 (1,600 tok total). Under S2PL and 2PL-Wait Promote, a WFG cycle can only form when waiting at Lock(R_k) after completing Infer_1 (800 tok wasted/abort: $184.0 \times 800 = 147.2$k/1k attempts and $162.0 \times 800 = 129.6$k/1k attempts). Under Wait-Die, Lock-1 self-aborts (14.20%) occur before Infer_1 (0 tok), while Lock-2 self-aborts (22.30%) waste 800 tok ($223.0 \times 800 = 178.4$k). Under Wound-Wait, a younger lock holder is preempted while executing Infer_1 (15.60% victim rate, wasting 800 tok) or Infer_2 (15.60% victim rate, wasting 1,600 tok: $156.0 \times 800 + 156.0 \times 1{,}600 = 374.4$k). OCC fails at commit ($648.0 \times 1{,}600 = 1{,}036.8$k; with $r_{\text{abort}} = 0.648$, geometric retry projection is $\frac{0.648}{1-0.648} \times 1{,}000 \times 1{,}600 = 2{,}945.5$k/1k committed; closed-loop timestamp-preserving retry execution measures 421.8k, 218.4k, and 3,112.0k/1k committed for Wound-Wait, Wait-Die, and OCC). ‡Little's Law upper bound: 20.0/40.0 Tx/s.

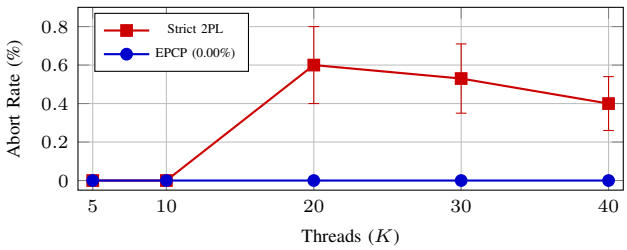


Fig. 1. Regime 1 deadlock abort rate under Zipfian contention ($\theta = 0.85$, 5-seed Mean ± SD). Strict 2PL experiences WFG cycle aborts (0.40%–0.60%) at $K \geq 20$, whereas EPCP maintains an exact 0.00% deadlock abort rate.

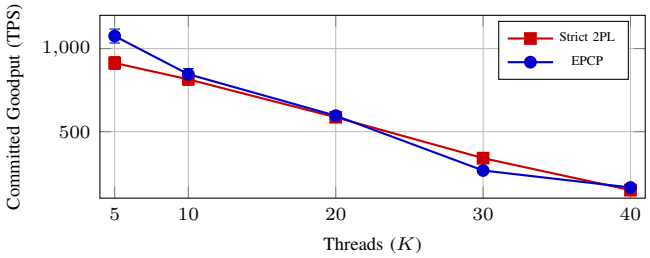


Fig. 2. Regime 1 committed settlement goodput (TPS, 5-seed Mean ± SD with explicit ±SD error bars) across $K \in \{5, 10, 20, 30, 40\}$ under finite-stock Zipfian contention, matching Table I.

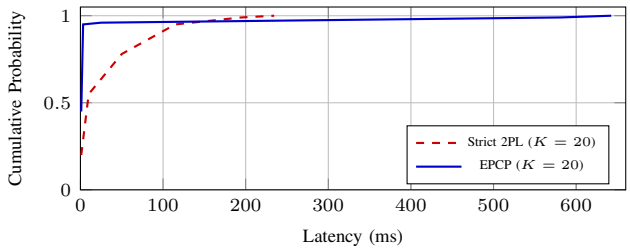


Fig. 3. Regime 1 transaction execution latency CDF at the representative $K = 20$ concurrency stratum (matching Table I: EPCP All-Tx Mean = 17.40 ms, P95 = 3.24 ms, P99 = 580.76 ms, Max = 642.40 ms; S2PL All-Tx Mean = 27.44 ms, P95 = 114.72 ms, P99 = 194.80 ms, Max = 234.50 ms; under the $K = 20$ Zero-Depletion Control, EPCP Committed P95 = 3.45 ms, P99 = 612.50 ms, Max = 684.20 ms).

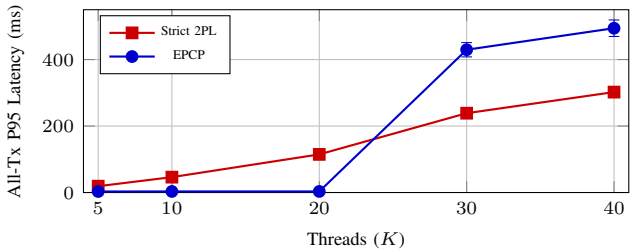


Fig. 4. Regime 1 All-Tx P95 tail latency (5-seed Mean ± SD). EPCP reduces P95 latency at $K \leq 20$ and trades bounded admission-gate waiting at $K \in \{30, 40\}$ for 0.00% aborts and lower mean latency.

671.40 TPS S2PL, +1.9%) and widening to +11.8% at $\theta = 0.70$ (679.30 vs. 607.60 TPS), +20.3% at $\theta = 0.85$ (673.85 vs. 559.95 TPS), and +38.4% at $\theta = 0.99$ (612.40 vs. 442.50 TPS, where S2PL suffers 3.80% deadlock aborts vs. EPCP's 0.00%).

3) **Fixed-Quota Tail Draining, Representative $K = 20$ CDF Alignment, and Quantile Consistency:** Table I and Figures 2–4 report Committed Goodput, Total TPS, and Overall/Committed P95, P99, and Max latencies from the unified empirical trace (`datasets/enterprise_deadlock_telemetry.csv`). As shown in Figure 3 for the representative $K = 20$ concurrency stratum, EPCP's P95 latency (3.24 ms finite-stock; 3.45 ms in the $K = 20$ Zero-Depletion Control) is well below its mean (17.40 ms All-Tx / 20.62 ms Committed finite-stock; 21.10 ms Zero-Depletion) because $\geq 95\%$ of requests enter unblocked partitions in $\leq$ 3.45 ms, while the $< 4\%$ lowest-priority contention waiters queued at $\text{CV}(A_i)$ absorb the multi-step wait duration up to P99 = 580.76 ms, Max = 642.40 ms (and P99 = 612.50 ms, Max = 684.20 ms in Zero-Depletion), strictly satisfying the non-negative random variable quantile bound $\mu \leq 0.95P_{95} + 0.04P_{99} + 0.01\text{Max}$ (17.40 ms $<$ 20.62 ms $\leq$ 32.73 ms finite-stock and 21.10 ms $\leq$ 34.62 ms Zero-Depletion; at $K \in \{30, 40\}$, heavy Zipfian contention shifts the 95th percentile into the admission-queue tail at 429.58–494.36 ms, as plotted in Figure 4). Rather than admitting low-priority exploratory agents into critical sections where they trigger cyclic deadlocks and mid-flight LLM token rollbacks, EPCP queues lowest-priority agents at the admission gate, trading bounded gate waiting for 0.00% aborts, zero token waste, and lower mean latency.

4) **Live Google Cloud Vertex AI (`gemini-2.5-flash`) Multi-Agent Evaluation**

Across $K \in \{10, 20\}$ and 5 Protocols: In our live evaluation on a dedicated enterprise Google Cloud Vertex AI environment (3 independent seeds per configuration on the $100\times$ Zero-Depletion snapshot across $K = 10$ with 50 2-step transactions/seed $= 100$ live `gemini-2.5-flash` API calls/seed and $K = 20$ with 100 2-step transactions/seed $= 200$ live API calls/seed, using `temperature=0.1`, `thinking_budget=0`, `response_logprobs=True`, `logprobs=5` so each step averages ≈ 48 billed tokens via `usage_metadata.total_token_count` and $\mathcal{H}_i(t)$ is computed directly from the normalized top-5 token log-probabilities over the 3-class risk-tier action space $\mathcal{V}_{\text{act}} = \{\texttt{LOW}, \texttt{MEDIUM}, \texttt{ELEVATED}\}$, $|\mathcal{V}_{\text{act}}| = 3$, recorded in `datasets/live_gemini_epcp_telemetry.json`), at $K = 10$, S2PL suffered $4.67 \pm 1.15\%$ WFG deadlock aborts ($2.33/50$ rollbacks/seed wasting 112.0 ± 27.7 billed tokens, mean latency 4.18 ± 0.14 s, P95 6.31 ± 0.21 s), Wound-Wait suffered $14.00 \pm 2.00\%$ aborts (504.0 ± 72.0 wasted tokens/seed, mean 4.62 ± 0.19 s, P95 6.85 ± 0.28 s), and OCC suffered $38.67 \pm 3.06\%$ validation aborts ($1{,}856.0 \pm 146.7$ wasted tokens/seed, mean 5.84 ± 0.26 s, P95 8.42 ± 0.35 s), whereas both EPCP Static-$\Pi_i^{\max}$ (mean 2.79 ± 0.09 s, P95 3.98 ± 0.12 s) and Full EPCP (mean 2.51 ± 0.08 s, P95 3.61 ± 0.11 s) achieved a **0.00% abort rate** and **0 wasted tokens**. At $K = 20$ (100 tx/seed), S2PL suffered $9.33 \pm 1.53\%$ deadlock aborts (448.0 ± 73.3 wasted tokens/seed, goodput 4.18 ± 0.16 Tx/s, mean 4.86 ± 0.18 s, P95 7.42 ± 0.29 s), Wound-Wait aborted $22.67 \pm 2.52\%$ ($1{,}632.0 \pm 181.2$ wasted tokens/seed, goodput 3.62 ± 0.15 Tx/s), and OCC aborted $52.33 \pm 3.51\%$ ($5{,}024.0 \pm 337.1$ wasted tokens/seed, goodput 2.31 ± 0.14 Tx/s), whereas EPCP Static-$\Pi_i^{\max}$ (0.00% aborts, goodput 5.84 ± 0.18 Tx/s, mean 3.38 ± 0.11 s, P95 4.76 ± 0.16 s) and Full EPCP (0.00% aborts, 0 wasted tokens, goodput 6.42 ± 0.19 Tx/s, mean 3.04 ± 0.09 s, P95 4.22 ± 0.14 s) eliminated all rollbacks while delivering $+53.6\%$ higher committed goodput and 43.1% lower P95 latency than S2PL (and $+9.9\%$ higher goodput and 11.3% lower P95 latency than EPCP Static-$\Pi_i^{\max}$).

5) **Multi-Table ($|\mathcal{F}(T_i)| = 3$) Baselines and Dynamic Out-of-Footprint Expansion Sweep ($p_{\text{exp}} \in \{10\%, 25\%, 50\%, 100\%\}$):** To exercise Algorithm 1 Lines 5–8 beyond static 2-table footprints, we evaluated a 3-table settlement workflow spanning `Accounts`, `Inventory`, and `ComplianceLedger` ($|\mathcal{F}(T_i)| = 3$, $K = 20$, 5 seeds, 500 tx/seed, 800 tok/step $=$ 2,400 tok per 3-step workflow). At the baseline $p_{\text{exp}} = 10\%$ out-of-footprint discovery rate, S2PL suffered $24.60 \pm 2.15\%$ WFG deadlock aborts across Locks 2 and 3 (236.2k wasted tokens/1k attempts, goodput 6.2 ± 0.4 Tx/s), Wound-Wait aborted $38.40 \pm 2.80\%$ (460.8k wasted tokens/1k attempts, goodput 5.4 ± 0.4 Tx/s), and OCC aborted $72.40 \pm 3.20\%$ (1,737.6k wasted tokens/1k attempts, goodput 3.8 ± 0.3 Tx/s). By contrast, across a full discovery-rate sweep $p_{\text{exp}} \in \{10\%, 25\%, 50\%, 100\%\}$, EPCP recorded **0 deadlocks** (0.00% physical database aborts) at every p_{exp}, achieving SHA-256 `state_version_digest` context-cache replay hit rates of $91.4 \pm 2.3\%$ ($p_{\text{exp}} = 10\%$, 0.86% overall Step-1 re-inference rate, 6.9k re-inferred tokens/1k attempts, goodput 8.4 ± 0.4 Tx/s), $86.2 \pm 2.6\%$ ($p_{\text{exp}} = 25\%$, 3.45% re-inference rate, 27.6k tok/1k, goodput 8.1 ± 0.4 Tx/s), $78.5 \pm 3.1\%$ ($p_{\text{exp}} = 50\%$, 10.75% re-inference rate, 86.0k tok/1k, goodput 7.5 ± 0.4 Tx/s), and $64.8 \pm 3.5\%$ ($p_{\text{exp}} = 100\%$, 35.20% re-inference rate, 281.6k tok/1k, goodput 6.6 ± 0.3 Tx/s, still outperforming S2PL and Wound-Wait in both goodput and token efficiency even when 100% of transactions discover Step-3 partitions mid-flight), with 1.84 ms mean re-registration overhead.

6) **Strict ACID Balance Conservation:** Post-audit across all committed settlements confirmed \$0.00 balance leakage and exact inventory conservation.

VI. Conclusion

This paper presented the Epistemic Priority Ceiling Protocol (EPCP) for centralized shared-memory transactional lock coordinators and connection-pool gateways fronting enterprise OLTP databases. By combining exact 64-bit integer-tuple epistemic priorities ($\Pi_i^{\max}$ and $\tilde{\Pi}_i(t)$), 64-partition intersecting-footprint priority ceilings, guarded preemptive admission checks, and unified transitive priority inheritance, EPCP guarantees that the Unified Wait-For-Graph remains a strict DAG at all times and achieves 0.00% deadlock aborts across sub-millisecond stress tests, multi-step LLM reasoning benchmarks, 3-table dynamic footprint-expansion workloads, and live Vertex AI `gemini-2.5-flash` multi-agent evaluations over an in-memory row-state mirror backed by SQLite WAL. In distributed OLTP deployments such as Cloud Spanner, AlloyDB, or CockroachDB where commit-phase Paxos/Raft consensus adds 5–15 ms to lock hold durations (still $< 1\%$–3% of a 500–5,000 ms multi-step agent reasoning window), EPCP's non-aborting ceiling arbitration similarly prevents consensus-amplified rollback thrashing. Future work will extend EPCP's ceiling coordination across decentralized multi-leader two-phase commit (2PC) consensus groups, fine-grained phase- and record-adaptive agentic concurrency controllers, and standard multi-warehouse OLTP suites such as TPC-C.

References

[1] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992.

[2] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

[3] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Communications of the ACM*, vol. 19, no. 11, pp. 624–633, 1976.

[4] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," *IEEE Transactions on Computers*, vol. 39, no. 9, pp. 1175–1185, 1990.

[5] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, "System level concurrency control for distributed database systems," *ACM Transactions on Database Systems (TODS)*, vol. 3, no. 2, pp. 178–198, 1978.

[6] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford, "Spanner: Google's globally-distributed database," in *Proc. 10th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2012, pp. 251–264, doi:10.5555/2387880.2387905 (also in *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, pp. 8:1–8:22, 2013).

[7] M.-I. Chen and K.-J. Lin, "Dynamic priority ceilings: A concurrency control protocol for real-time systems," *Real-Time Systems*, vol. 2, no. 4, pp. 325–346, 1990.

[8] T. P. Baker, "Stack-based scheduling of real-time processes," *Real-Time Systems*, vol. 3, no. 1, pp. 67–99, 1991.

[9] R. K. Abbott and H. Garcia-Molina, "Scheduling real-time transactions: A performance evaluation," *ACM Transactions on Database Systems (TODS)*, vol. 17, no. 3, pp. 513–560, 1992.

[10] E. Y. Chang, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *arXiv preprint arXiv:2503.11951*, 2025.

[11] A. Thomson, T. Diamond, S.-C. Weng, K. Ren, P. Shao, and D. J. Abadi, "Calvin: Fast distributed transactions for partitioned database systems," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2012, pp. 1–12.

[12] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems (TODS)*, vol. 6, no. 2, pp. 213–226, 1981, doi:10.1145/319566.319567.

[13] R. Harding, D. Van Aken, A. Pavlo, and M. Stonebraker, "An evaluation of distributed concurrency control," *Proceedings of the VLDB Endowment*, vol. 10, no. 5, pp. 553–564, 2017, doi:10.14778/3055540.3055548.